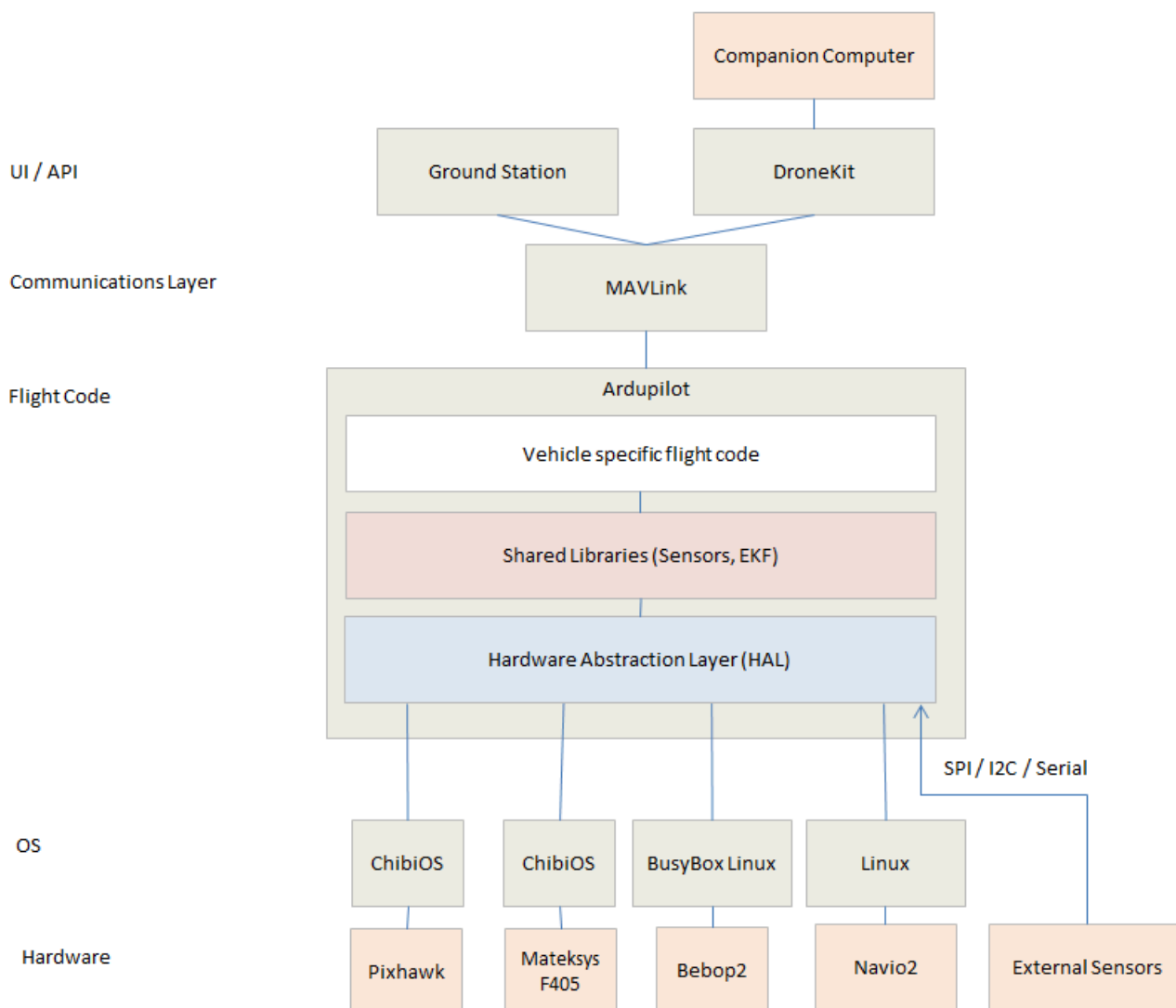


Learning ArduPilot — Introduction

This page introduces the basic structure of ArduPilot. Before you get started you should work out what code exploring system you will use. You could just use a web browser and look at <https://github.com/ArduPilot/ardupilot/> but you will probably get a lot more out of it if you have [cloned all of the git repositories](#) and use a good programmer's IDE like the ones recommended [here](#).

Basic structure ¶



The basic structure of ArduPilot is broken up into 5 main parts:

- vehicle code
- shared libraries

- hardware abstraction layer (AP_HAL)
- tools directories
- external support code (e.g. mavlink)

Vehicle Code

The vehicle directories are the top level directories that define the firmware for each vehicle type. Currently, there are 6 vehicle types: Plane, Copter, Rover, Sub, Blimp and AntennaTracker. Although there are a lot of common elements between different vehicle types, they are each different. For now, we only have a [detailed description of the code structure for the Copter code](#).

Along with the *.cpp files, each vehicle directory contains a wscript file which lists library dependencies.

Libraries

The [libraries](#) are shared amongst all vehicle types. These libraries include sensor drivers, attitude and position estimation (aka [EKF](#)) and control code (i.e. PID controllers). See the [Library Description](#), [Library Example Sketches](#) and [Sensor Drivers](#) pages for more details.

AP_HAL

The AP_HAL layer (Hardware Abstraction Layer) is how we make ArduPilot portable to lots of different platforms. There is a top-level AP_HAL in libraries/AP_HAL that defines the interface that the rest of the code has to specific board features, and then there is a AP_HAL_XXX subdirectory for each board type, for example, AP_HAL_ChibiOS for stm32-based boards, AP_HAL_ESP32 for ESP32 boards and AP_HAL_Linux for Linux based boards.

Tools directories

The tools directories are miscellaneous support directories. For example, tools/autotest provides the autotest infrastructure behind the autotest.ardupilot.org site and tools/Replay provides our log replay utility.

External support code

On some platforms we need external support code to provide additional features or board support. Currently the external trees are:

- [ChibiOS](#) - the ChibiOS RTOS used on stm32-based boards
- [DroneCAN](#) - the CANBUS implementation used in ArduPilot
- [mavlink](#) - the mavlink protocol and code generator

Note

Most of these are imported as [Git Submodules](#) when you [build ArduPilot](#).

ArduPilot Libraries

The [libraries](#) are shared with Copter, Plane and Rover. Below is a high level list of libraries and their function.

Core libraries:

- [AP_AHRS](#) - attitude estimation using DCM or EKF
- [AP_Common](#) - core includes required by all sketches and libraries
- [AP_Math](#) - various math functions especially useful for vector manipulation
- [AC_PID](#) - PID(Proportional-Integral-Derivative) controller library
- [AP_InertialNav](#) - inertial navigation library for blending accelerometer inputs with gps and baro data
- [AC_AttitudeControl](#) - ArduCopter's control library includes various functions of attitude, position control based on PID control.
- [AC_WPNav](#) - waypoint navigation library
- [AP_Motors](#) - multicopter and traditional helicopter motor mixing
- [RC_Channel](#) - a library to more convert pwm input/output from APM_RC into internal units such as angles
- [AP_HAL](#), [AP_HAL_ChibiOS](#), [AP_HAL_Linux](#) - libraries to implement the "Hardware abstraction layer" which presents an identical interface to the high level code so that it can more easily be ported to different boards.

Sensor libraries:

- [AP_InertialSensor](#) - reads gyro and accelerometer data, perform calibration and provides data in standard units (deg/s, m/s) to main code and other libraries
- [AP_RangeFinder](#) - sonar and ir distance sensor interfaced library
- [AP_Baro](#) - barometer interface library
- [AP_GPS](#) - gps interface library
- [AP_Compass](#) - 3-axis compass interface library
- [AP_OpticalFlow](#) - optical flow sensor interface library

Other libraries:

- [AP_Mount](#), [AP_Camera](#), [AP_Relay](#) - camera mount control library, camera shutter control libraries

- [AP_Mission](#) - stores/retrieves mission commands from eeprom

Library Example Sketches

The first step in exploring the code for yourself is to use the example sketches for the libraries. Following the arduino tradition we have example sketches for most libraries. A 'sketch' is just a main program, written as a cpp file.

Knowing the library API and conventions used in ArduPilot is essential to understanding the code. So using the library example sketches is a great way to get started. As a start you should read, build and run the example sketches for the following libraries:

- `libraries/AP_GPS/examples/GPS_AUTO_test`
- `libraries/AP_InertialSensor/examples/INS_generic`
- `libraries/AP_Compass/examples/AP_Compass_test`
- `libraries/AP_Baro/examples/BARO_generic`
- `libraries/AP_AHRS/examples/AHRS_Test`

For example, the following will build and install the AP_GPS example sketch on a Pixhawk:

```
cd $ARDUPILOT_HOME # the top-level of an ArduPilot repository
./waf configure --board=Pixhawk1
./waf build --target examples/INS_generic --upload
```

waf can list the examples it can build:

```
cd $ARDUPILOT_HOME
./waf list | grep 'examples'
```

Once you have uploaded the example you can look at the output by attaching to the console. What the console is depends on the type of board. On Pixhawk boards it is the USB connector. So just connect to the USB device with your favourite serial program (the baudrate doesn't matter).

For example, if you have mavproxy installed, you could do this to connect to a Pixhawk on Linux:

```
mavproxy.py --setup --master /dev/serial/by-id/usb-3D_Robotics_PX4_FMU_v2.x_0-if00
```

Using the `--setup` option puts mavproxy into raw serial mode, instead of processed MAVLink mode. That is what you need for the example sketches.

Running Examples in SITL

Certain sketches can also be run in SITL. For example to run the protocol decoder sketch:

```
cd $ARDUPILOT_HOME # the top-level of an ArduPilot repository
./waf configure --board sitl
./waf build --target examples/RCProtocolDecoder
```

To start the sketch, run it directly:

```
./build/sitl/examples/RCProtocolDecoder -M quad -C
```

Understanding the example sketch code

When you are reading the example sketch code (such as the [GPS_AUTO_test](#) code) you will notice a few things that may seem strange at first:

- it declares a 'hal' variable as a reference
- the code is quite rough and not well commented
- the `setup()` and `loop()` functions

The hal reference

Every file that uses features of the AP_HAL needs to declare a hal reference. That gives access to a `AP_HAL::HAL` object, which provides access to all hardware specific functions, including things like printing messages to the console, sleeping and talking to I2C and SPI buses.

The actual hal variable is buried inside the board specific `AP_HAL_XXX` libraries. The reference in each file just provides a convenient way to get at the hal.

The most commonly used hal functions are:

- `hal.console->printf()` to print strings
- `AP_HAL::millis()` and `AP_HAL::micros()` to get the time since boot

- `hal.scheduler->delay()` and `hal.scheduler->delay_microseconds()` to sleep for a short time
- `hal.gpio->pinMode()`, `hal.gpio->read()` and `hal.gpio->write()` for accessing GPIO pins
- I2C access via `hal.i2c`
- SPI access via `hal.spi`

Go and have a look in the `libraries/AP_HAL` directory now for the full list of functions available on the HAL.

The `setup()` and `loop()` functions

You will notice that every sketch has a `setup()` function and `loop()` function. The `setup()` function is called when the board boots. The actual call comes from within the HAL for each board, so the `main()` function is buried inside the HAL, which then calls `setup()` after board specific startup is complete.

The `setup()` function is only called once, and typically initialises the libraries, and maybe prints a “hello” banner to show it is starting.

After `setup()` is finished the `loop()` function is continuously called (by the main code in the `AP_HAL`). The main work of the sketch is typically in the `loop()` function.

Note that this `setup()/loop()` arrangement is only the tip of the iceberg for more complex boards. It may make it seem that ArduPilot is single threaded, but in fact there is a lot more going on underneath, and on boards that have threading (such as Pixhawk and Linux based boards) there will in fact be lots of realtime threads started. See the section on understanding ArduPilot threading below.

The `AP_HAL_MAIN()` macro

You will notice a extra line like this at the bottom of every sketch:

```
AP_HAL_MAIN();
```

That is a HAL macro that produces the necessary code to declare a C++ main function, along with any board level initialization code for the HAL. You rarely have to worry about how it works, but if you are curious you can look for the `#define` in the `AP_HAL_XXX` directories in each HAL. It is usually in `AP_HAL_XXX_Main.h`.

Rough Example code

You will notice that the example sketches are quite rough, and badly commented. This is your opportunity to make a contribution to the code! As you read through the example sketches and explore how they work add some comments to the code to explain the APIs and then [submit a pull request](#) so others can benefit from your study.

Sensor Drivers

ArduPilot supports a wide variety of sensors from many different manufacturers. One clear example of this can be seen in the [list of range finders](#) (aka sonars, lidars). This page attempts to explain how sensor drivers are written and integrated into the vehicle code.

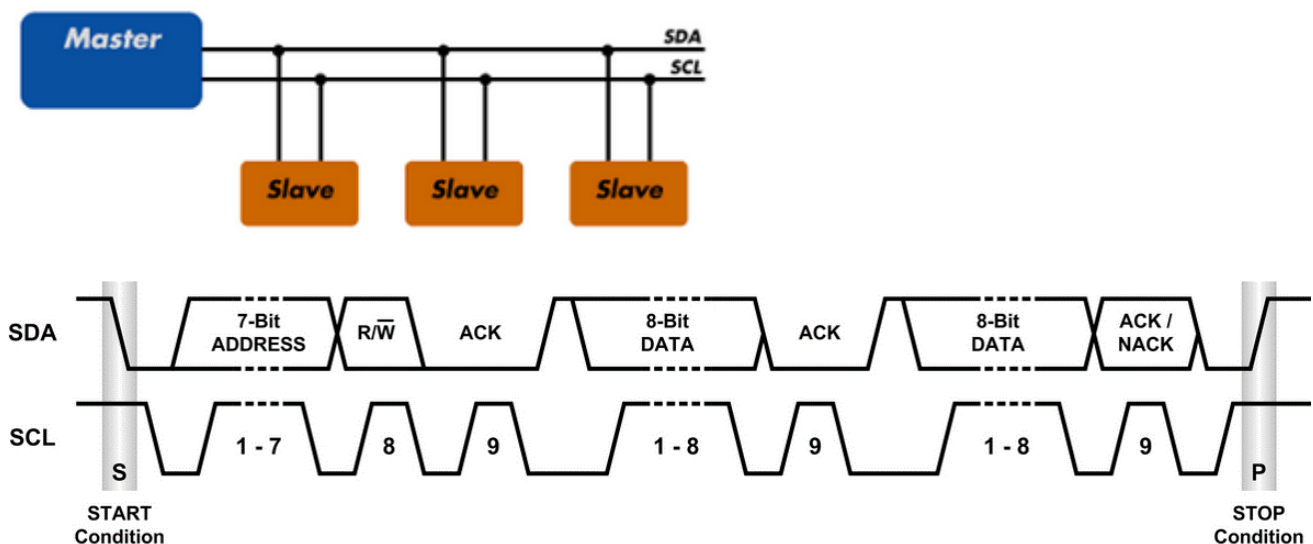
Supported Protocols

[I2C](#), [SPI](#), [UART \(aka Serial\)](#) and [CANBUS](#) (in particular UAVCAN/DroneCAN) protocols are supported.

If you plan to write a new driver, you will likely need to refer to the sensor's datasheet in order to determine which protocol it uses. Alternatively, you could utilize an intermediary node to move the necessary data back to the autopilot.

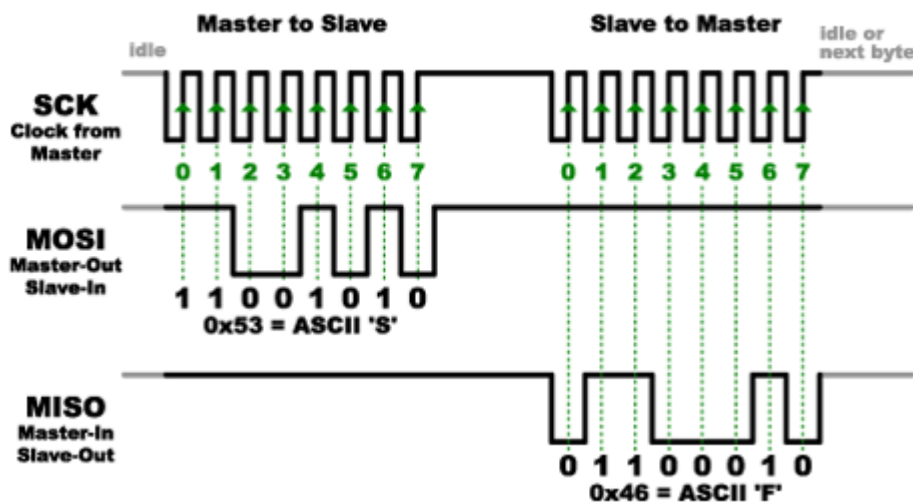
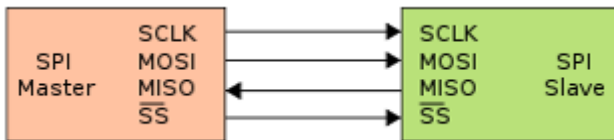
I2C

- one master, many slaves possible
- a relatively simple protocol which is good for communicating over short-distances (i.e. less than 1m).
- bus runs at 100kHz or 400kHz but the data rate is relatively low compared to other protocols.
- only 4 pins are required (VCC, GND, SDA, SCL)



SPI

- one master, one slave
- 20Mhz+ speed meaning it is very fast especially compared to I2C
- only works over short distances (10cm)
- requires at least 5 pins (VCC, GND, SCLK, Master-Out-Slave-In, Master-In-Slave-Out) + 1 slave select pin per slave

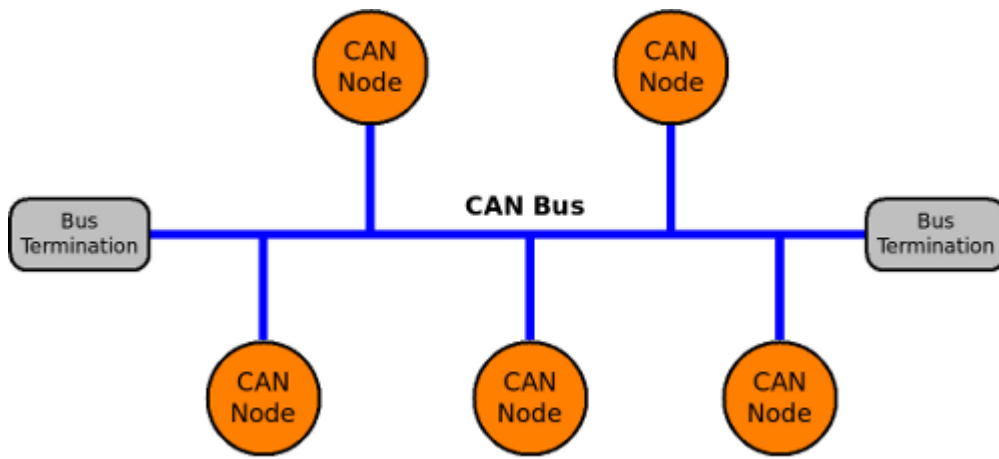


Serial / UART

- one master, one slave
- character based protocol good for communicating over longer distances compared to I2C and SPI (i.e. 1m)
- relatively fast at 57Kbps ~ 1.5Mbps
- at least 4 pins required (VCC, GND, TX, RX), plus 2 optional pins (Clear-To-Send, Clear-To-Receive)

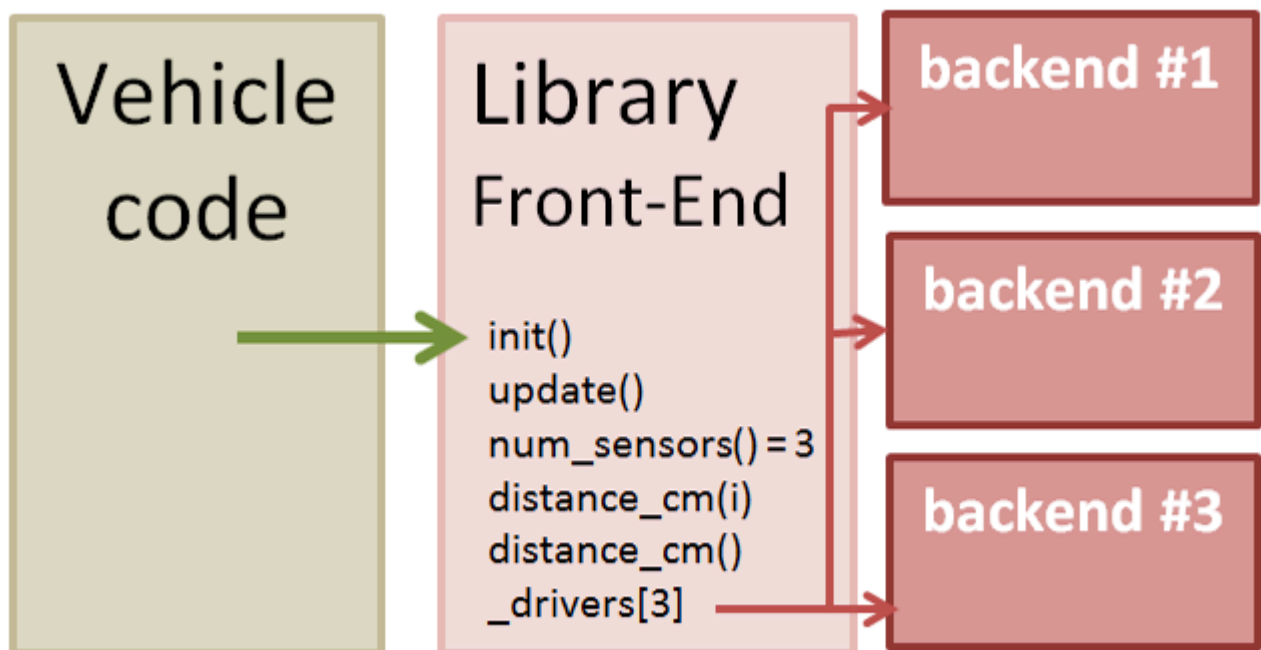
CAN bus with UAVCAN

- multimaster bus, any node can initiate transmission of data when they need to
- packet based protocol for very long distances
- high speed, typically 1 Mb (however only 50% of the bus bitrate can really be used without major collisions)
- at least 3 pins required (GND, CAN HI, CAN LO). Optionally VCC can be used to power nodes
- point-to-point topology. Star or stubs topology is not advised
- termination is required at each end of the bus



FrontEnd / BackEnd Split

An important concept within the sensor driver architecture is the front-end / back-end split.

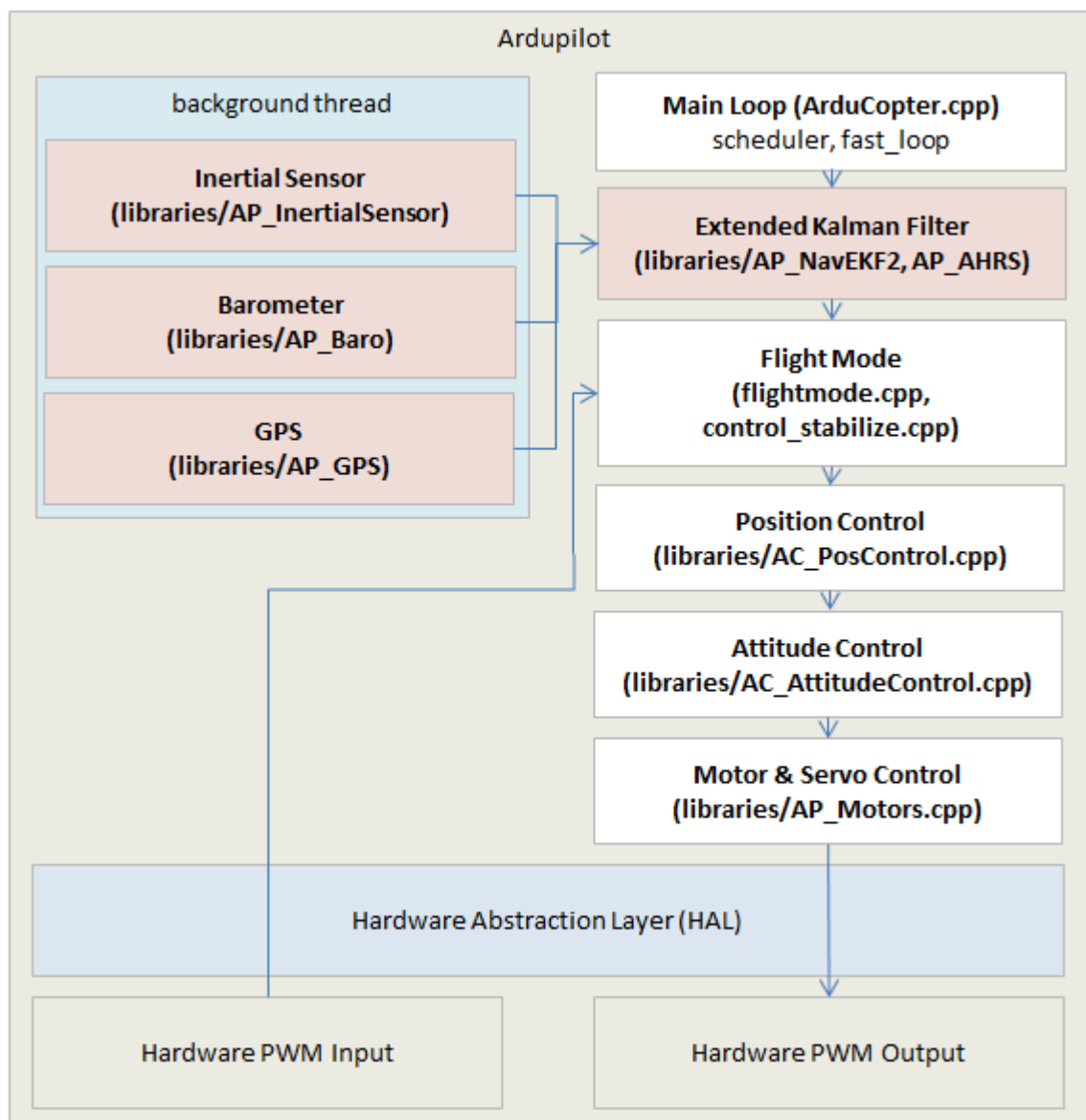


The vehicle code only ever calls into the Library's (aka sensor driver's) front-end.

On start-up the front-end creates one or more back-ends based either on automatic detection of the sensor (i.e. probing for a response on a known I2C address) or by using the user defined `_TYPE` params (i.e. `RNGFND_TYPE`, `RNGFND_TYPE2`). The front-end maintains pointers to each back-end which are normally held within an array named `_drivers[]`.

User settable parameters are always held within the front-end.

How and When the driver code is run



The image above shows a zoomed in view of the ardupilot architecture. The top-left blue box illustrates how the sensor driver's back-ends are run in a background thread. Raw data from the sensors is collected, converted into standard units and then held within buffers within the driver.

The vehicle code's main thread runs regularly (i.e. 400hz for copter) and accesses the latest data available through methods in the driver's front-end. For example in order to calculate the latest attitude estimate, the AHRS/EKF would pull the latest accelerometer, gyro and compass information from the sensor drivers' front-ends.

The image is a slight generalization, for drivers using I2C or SPI, they must run in the background thread so that the highrate communication with the sensor does not affect the main loop's performance but for driver's using a serial (aka UART) interface, it is safe to run in the main thread because the underlying serial driver itself collects data in the background and includes a buffer.

Vehicle Code and Front-End Example

The example below shows how the Copter vehicle code pulls data from range finder (aka sonar, lidar) drivers. The Copter code's [scheduler](#) calls the vehicle's `read_rangefinder()` method at 20Hz. Below is a picture of this method, the latest version can be seen in the [sensors.cpp](#) file. The `rangefinder.update()` method is a call into the driver's front-end.

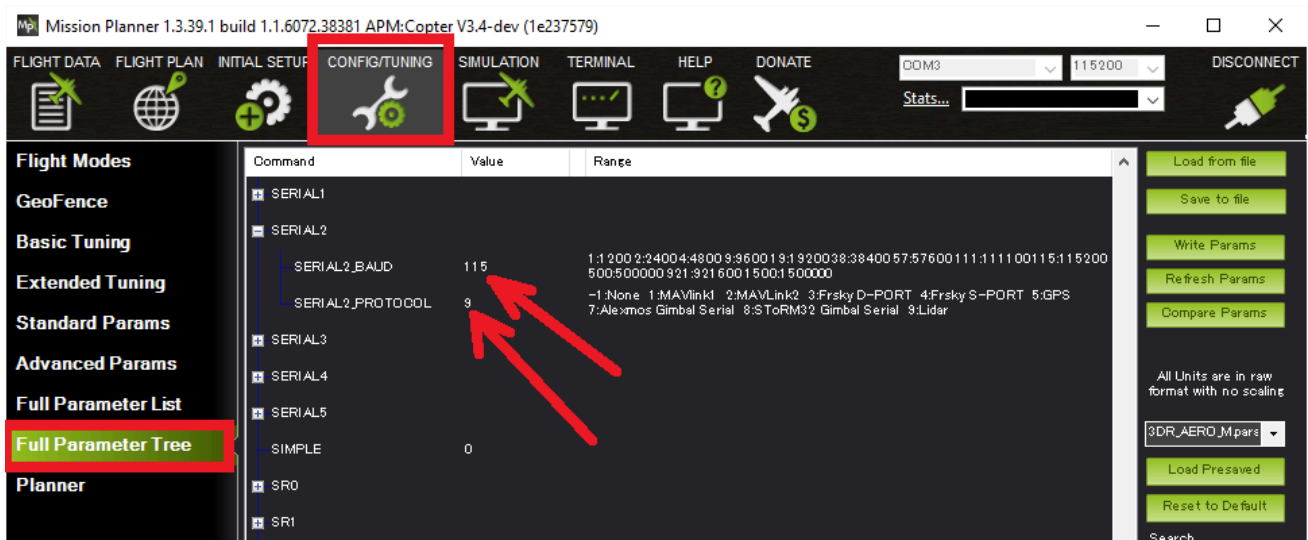
```
27 // return rangefinder altitude in centimeters
28 void Copter::read_rangefinder(void)
29 {
30     #if RANGEFINDER_ENABLED == ENABLED
31         rangefinder.update();
32     #endif
33     #if RANGEFINDER_TILT_CORRECTION == ENABLED
34         const float tilt_correction = MAX(0.707f, ahrs.get_rotation_body_to_ned().c.z);
35     #else
36         const float tilt_correction = 1.0f;
37     #endif
38
39     // iterate through downward and upward facing lidar
40     struct {
41         RangeFinderState &state;
42         enum Rotation orientation;
43     } rngfnd[2] = { {rangefinder_state, ROTATION_PITCH_270}, {rangefinder_up_state, ROTATION_PITCH_90}};
```

Below is the range finder driver's front-end [update method](#). This gives the driver a chance to do any general processing it might want to within the main thread. Each back-end's update method is called in turn.

```
285 /*
286     update RangeFinder state for all instances. This should be called at
287     around 10Hz by main loop
288 */
289 void RangeFinder::update(void)
290 {
291     for (uint8_t i=0; i<num_instances; i++) {
292         if (drivers[i] != nullptr) {
293             if ((Type)params[i].type.get() == Type::NONE) {
294                 // allow user to disable a rangefinder at runtime
295                 state[i].status = Status::NotConnected;
296                 state[i].range_valid_count = 0;
297                 continue;
298             }
299             drivers[i]->update();
300         }
301     }
```

UART/Serial Back-End Example

Next is the update method of the [LightWare back-end](#) using the serial protocol. As described on the [user wiki](#) the serial range finder can be connected to any of the flight controller's serial ports but the user must specify which serial port, and what baud rate is used by setting the `SERIALX_BAUD` and `SERIALX_PROTOCOL` parameters.



Within the serial driver's [backend code](#), it first finds which UART the user would like to use via the serial_manager class which looks for the parameters settings described above.

```

24  /*
25   The constructor also initialises the rangefinder. Note that this
26   constructor is not called until detect() returns true, so we
27   already know that we should setup the rangefinder
28  */
29  AP_RangeFinder_Backend_Serial::AP_RangeFinder_Backend_Serial(
30      RangeFinder::RangeFinder_State &_state,
31      AP_RangeFinder_Params &_params,
32      uint8_t serial_instance) :
33      AP_RangeFinder_Backend(_state, _params)
34  {
35      uart = AP::serialmanager().find_serial(AP_SerialManager::SerialProtocol_Rangefinder, serial_instance);
36      if (uart != nullptr) {
37          uart->begin(initial_baudrate(serial_instance), rx_bufsize(), tx_bufsize());
38      }
39  }

```

Get UART

Each time the driver's back-end update() method is called it calls the get_reading method which checks if new characters have arrived from the sensor and then decodes them.

As mentioned above, because the serial protocol implements its own buffering, the processing of any data (see get_reading method) from the sensor is run here in the main thread. I.e. there is no "register_periodic_callback" like you will see in I2C and SPI drivers.

```

56  /*
57  update the state of the sensor
58  */
59  void AP_RangeFinder_Backend_Serial::update(void)
60  {
61      if (get_reading(state.distance_cm)) {
62          // update range_valid state based on distance measured
63          state.last_reading_ms = AP_HAL::millis();
64          update_status();
65      } else if (AP_HAL::millis() - state.last_reading_ms > read_timeout_ms()) {
66          set_status(RangeFinder::Status::NoData);
67      }
68  }

```

```

50  // read - return last value measured by sensor
51  bool AP_RangeFinder_LightWareSerial::get_reading(uint16_t &reading_cm)
52  {
53      if (uart == nullptr) {
54          return false;
55      }
56
57      // read any available lines from the lidar
58      float sum = 0;
59      uint16_t count = 0;
60      int16_t nbytes = uart->available();  check if data is available
61      while (nbytes-- > 0) {
62          char c = uart->read();  read a byte
63          if (c == '\r') {
64              linebuf[linebuf_len] = 0;
65              sum += (float)atof(linebuf);  convert to number if necessary
66              count++;
67              linebuf_len = 0;
68          } else if (isdigit(c) || c == '.') {
69              linebuf[linebuf_len++] = c;
70              if (linebuf_len == sizeof(linebuf)) {
71                  // too long, discard the line
72                  linebuf_len = 0;
73              }
74          }
75      }
76
77      // we need to write a byte to prompt another reading
78      uart->write('d');  write byte to sensor
79
80      if (count == 0) {
81          return false;
82      }
83      reading_cm = 100 * sum / count;
84      return true;
85  }

```

I2C Back-End Example

This example shows the back-end for the Lightware I2C driver. In this case, the front-end gets the I2C bus and passes it to the back-end during initialisation.

libraries/AP_RangeFinder/RangeFinder.cpp

```
601 hal.i2c_mgr->get_device(HAL_RANGEFINDER_LIGHTWARE_I2C_BUS, _address[instance]));
```

← get I2C bus and specify sensor's I2C address on that bus

libraries/AP_RangeFinder/AP_RangeFinder_LightWareI2C.cpp

```
63 void AP_RangeFinder_LightWareI2C::init()
```

```
64 {
```

```
65     // call timer() at 20Hz
```

```
66     _dev->register_periodic_callback(50000,
```

```
67         FUNCTOR_BIND_MEMBER(&AP_RangeFinder_LightWareI2C::timer, bool));
```

```
68 }
```

```
69
```

```
70 // read - return last value measured by sensor
```

```
71 bool AP_RangeFinder_LightWareI2C::get_reading(uint16_t &reading_cm)
```

```
72 {
```

```
73     be16_t val;
```

```
74
```

```
75     if (ranger._address[state.instance] == 0) {
```

```
76         return false;
```

```
77     }
```

```
78
```

```
79     // read the high and low byte distance registers
```

```
80     bool ret = _dev->read((uint8_t *) &val, sizeof(val));
```

```
81     if (ret) {
```

```
82         // combine results into distance
```

```
83         reading_cm = be16toh(val);
```

```
84     }
```

```
85
```

```
86     return ret;
```

```
87 }
```

← register call back (50000us = 50ms = 20hz)

← read a byte from sensor

← convert to cm

The back-end's init method then registers its "timer" method to be called at 20hz. Within the timer method (not shown) the get_reading() method is called which reads bytes from the sensor and converts the distance to centimeters.

SPI Back-End Example

This example shows the back-end for the MPU9250 IMU which includes a gyro, accelerometer and compass. The front-end gets the SPI bus and passes it to the back-end during initialisation.

```
libraries/AP_InertialSensor/AP_InertialSensor.cpp
```

```
0 AP_InertialSensor::detect_backends(void)
```

```
1 {
```

```
2     _add_backend(AP_InertialSensor_MPU9250::probe(*this, hal.spi->get_device(HAL_INS_MPU9250_NAME)));
```

← get SPI bus

the start() method is called during initialisation and configures the sensor. It uses semaphores to ensure no interference with other SPI devices on the same bus.

The _read_sample method is registered so that it is called at 1000hz. Note there is no need to take/give semaphores within the _read_sample method because that is done as part of the periodic callback code.

the _block_read method shows how data can be read from the sensor's registers.

/libraries/AP_InertialSensor/AP_InertialSensor_MPU9250.cpp

```
639 bool AP_InertialSensor_MPU9250::_block_read(uint8_t reg, uint8_t *buf, uint32_t size)
641 {
642     return _dev->read_registers(reg, buf, size); ← read bytes from sensor
643 }

296 void AP_InertialSensor_MPU9250::start()
297 {
298     if (!_dev->get_semaphore()->take(0)) { ← get semaphore
299         return;
300     }
302     // initially run the bus at low speed
303     _dev->set_speed(AP_HAL::Device::SPEED_LOW); ← set bus speed
359     _dev->get_semaphore()->give(); ← return semaphore
370     // start the timer process to read samples
371     _dev->register_periodic_callback(1000, FUNCTOR_BIND_MEMBER(&AP_InertialSensor_MPU9250::_read_sample, bool)); ← setup background read
372 }

558 bool AP_InertialSensor_MPU9250::_read_sample()
559 {
560     uint8_t n_samples;
561     uint16_t bytes_read;
562     uint8_t *rx = _fifo_buffer;
563     bool need_reset = false;
564
565     if (!_block_read(MPUREG_FIFO_COUNTH, rx, 2)) {
566         goto check_registers;
567     }
568
569     bytes_read = uint16_val(rx, 0); ← decode data from sensor
570     n_samples = bytes_read / MPU_SAMPLE_SIZE;
```

Additional Advice

When writing a sensor driver, never include any wait or sleep code because this will either delay the main thread or the background thread associated with the bus being used.

If a new library is written, it must be added to the wscript file in the vehicle directory (i.e. /ardupilot/ArduCopter/wscript) in order for it to be linked into the final binary

Threading

Understanding ArduPilot threading

Once you have learned the basic of the ArduPilot libraries it is time for you to understand how ArduPilot deals with threading. The `setup()/loop()` structure that was inherited from arduino may make it seem that ArduPilot is a single threaded system, but in fact it isn't.

The threading approach in ArduPilot depends on the board it is built for. Some boards (such as the APM1 and APM2) don't support threads, so make do with a simple timer and callbacks. Some boards (PX4 and Linux) support a rich Posix threading model with realtime priorities, and these are used extensively by ArduPilot.

There are a number of key concepts related to threading that you need to understand in ArduPilot:

- The timer callbacks
- HAL specific threads
- driver specific threads
- ardupilot drivers versus platform drivers
- platform specific threads and tasks
- the AP_Scheduler system
- semaphores
- lockless data structures

The timer callbacks

Every platform provides a 1kHz timer in the AP_HAL. Any code in ArduPilot can register a timer function which is then called at 1kHz. All registered timer functions are called sequentially. This very primitive mechanism is used as it is extremely portable, and yet very useful. You register a timer callback by calling the `hal.scheduler->register_timer_process()` like this:

```
hal.scheduler->register_timer_process(AP_HAL_MEMBERPROC(&AP_Baro_MS5611::_update));
```

that particular example is from the MS5611 barometer driver. The `AP_HAL_MEMBERPROC()` macro provides a way to encapsulate a C++ member function as a callback argument (bundling up the object context with the function pointer).

When a piece of code wants something to happen at less than 1kHz then it should maintain its own “last_called” variable and return immediately if not enough time has passed. You can use the `hal.scheduler->millis()` and `hal.scheduler->micros()` functions to get the time since boot in milliseconds and microseconds to support this.

You should now go and modify an existing example sketch (or create a new one) and add a timer callback. Make the timer increment a counter then print the value of the counter every second in the `loop()` function. Modify your function so that it increments the counter every 25 milliseconds.

HAL specific threads

On platforms that support real threads the `AP_HAL` for that platform will create a number of threads to support basic operations. For example, on Pixhawk the following HAL specific threads are created:

- The UART thread, for reading and writing UARTs (and USB)
- The timer thread, which supports the 1kHz timer functionality described above
- The IO thread, which supports writing to the microSD card, EEPROM and FRAM

Have a look in `Scheduler.cpp` inside each `AP_HAL` implementation to see what threads are created and what the realtime priority of each thread is.

If you have a Pixhawk then you should also now setup a debug console cable and attach it to the nsh console (the serial5 port). Connect at 57600. When you have connected, try the “ps” command and you will get something like this:

```
PID PRI  SCHD  TYPE NP  STATE NAME
0  0  FIFO  TASK  READY Idle Task()
1  192 FIFO  KTHREAD WAITSIG hpwork()
2  50  FIFO  KTHREAD WAITSIG lpwork()
3  100 FIFO  TASK  RUNNING init()
37 180 FIFO  TASK  WAITSEM AHRS_Test()
38 181 FIFO  PTHREAD WAITSEM <pthread>(20005400)
39 60  FIFO  PTHREAD READY <pthread>(20005400)
40 59  FIFO  PTHREAD WAITSEM <pthread>(20005400)
10 240 FIFO  TASK  WAITSEM px4io()
13 100 FIFO  TASK  WAITSEM fmuservo()
30 240 FIFO  TASK  WAITSEM uavcan()
```

In this example you can see the “AHRS_Test” thread, which is running the example sketch from `libraries/AP_AHRS/examples/AHRS_Test`. You can also see the timer thread (priority 181), the UART thread (priority 60) and the IO thread (priority 59).

Additionally you can see the px4io, fmservo, uavcan, lpwork, hpwork and idle tasks. More about those later.

Other AP_HAL ports have more or less threads depending on what is needed.

One common use of threads is to provide drivers a way to schedule slow tasks without interrupting the main autopilot flight code. For example, the AP_Terrain library needs to be able to do file IO to the microSD card (to store and retrieve terrain data). The way it does this is it calls the function `hal.scheduler->register_io_process()` like this:

```
hal.scheduler->register_io_process(AP_HAL_MEMBERPROC(&AP_Terrain::io_timer));
```

This sets up the `AP_Terrain::io_timer` function to be called regularly. That is called within the boards IO thread, meaning it is a low realtime priority and is suitable for storage IO tasks. It is important that slow IO tasks like this not be called on the timer thread as they would cause delays in the more important processing of high speed sensor data.

Driver specific threads

It is also possible to create driver specific threads, to support asynchronous processing in a manner specific to one driver. Currently you can only create driver specific threads in a manner that is platform dependent, so this is only appropriate if your driver is intended to run only on one type of autopilot board. If you want it to run on multiple AP_HAL targets then you have two choices:

- you can use the `register_io_process()` and `register_timer_process()` scheduler calls to use the existing timer or IO threads
- you can add a new HAL interface that provides a generic way to create threads on multiple AP_HAL targets (please contribute patches back)

An example of a driver specific thread is the ToneAlarm thread in the Linux port. See `AP_HAL_Linux/ToneAlarmDriver.cpp`

ArduPilot drivers versus platform drivers

You may notice some driver duplication in ArduPilot. For example, we have a MPU6000 driver in `libraries/AP_InertialSensor/AP_InertialSensor_MPU6000.cpp`, and another MPU6000 driver in `PX4Firmware/src/drivers/mpu6000`.

The reason for this duplication is that the PX4 project already provides a set of well tested drivers for hardware that comes with Pixhawk boards, and we enjoy a good collaborative relationship with the PX4 team on developing and enhancing these drivers. So when we build ArduPilot for PX4 we take advantage of the PX4 drivers by writing small “shim” drivers which present the PX4 drivers with the standard ArduPilot library interface. If you look at

libraries/AP_InertialSensor/AP_InertialSensor_PX4.cpp you will see a small shim driver that asks the PX4 what IMU drivers are available on this board and automatically makes all of them available as part of the ArduPilot AP_InertialSensor library.

So if we have an MPU6000 on the board we use the AP_InertialSensor_MPU6000.cpp driver on non-Pixhawk/NuttX platforms, and the AP_InertialSensor_PX4.cpp driver on NuttX based platforms.

The same type of split can also happen for other AP_HAL ports. For example, we could use Linux kernel drivers for some sensors on Linux boards. For other sensors we use the generic AP_HAL I2C and SPI interfaces to use the ArduPilot “in-tree” drivers which work across a wide range of boards.

Platform specific threads and tasks

On some platforms there will be a number of base tasks and threads that will be created by the startup process. These are very platform specific so for the sake of this tutorial I will concentrate on the tasks used on PX4 based boards.

In the “ps” output above we saw a number of tasks and threads that were not started by the AP_HAL_PX4 Scheduler code. Specifically they are:

- idle task - called when there is nothing else to run
- init - used to start up the system
- px4io - handle the communication with the PX4IO co-processor
- hpwork - handle thread based PX4 drivers (mainly I2C drivers)
- lpwork - handle thread based low priority work (eg. IO)
- fmuservo - handle talking to the auxiliary PWM outputs on the FMU
- uavcan - handle the uavcan CANBUS protocol

The startup of all of these tasks is controlled by the PX4 specific [rc.APM script](#). That script is run when the PX4 boots, and is responsible for detecting what sort of PX4 board we are using then loading the right tasks and drivers for that board. It is a “nsh” script, which is similar to a bourne shell script (though nsh is much more primitive).

As an exercise, try editing the rc.APM script and adding some sleep and echo commands. Then upload a new firmware and connect to the debug console while the board is booting. Your echo commands should show up on the console.

Another very useful way of exploring the startup of the PX4 is to boot without a microSD card in the slot. The [rcS script](#), which runs just before rc.APM, detects if a microSD is inserted and gives you a bare nsh console on the USB port if it isn't. You can then manually run all the steps of rc.APM yourself on the USB console to learn how it works.

Try the following exercise after booting a Pixhawk without a microSD card and connecting to the USB console:

```
tone_alarm stop
uorb start
mpu6000 start
mpu6000 info
mpu6000 test
mount -t binfs /dev/null /bin
ls /bin
perf
```

Try playing with the other drivers. Have a look in /bin to see what is available. The source code for most of these commands is in [PX4Firmware/src/drivers](#). Have a look through the mpu6000 driver to get an idea of what is involved.

Given we are on the topic of threads and tasks, a brief description of threads in the PX4Firmware git tree is worth mentioning. If you look in the mpu6000 driver you will see a line like this:

```
hrt_call_every(&_call, 1000, _call_interval, (hrt_callout)&MPU6000::measure_trampoline, this);
```

that is the equivalent of the `hal.scheduler->register_timer_process()` function in the AP_HAL, but is PX4 specific and is also much more flexible. It says that it wants the HRT (high resolution timer) subsystem of the PX4 to call the `MPU6000::measure_trampoline` function every 1000 microseconds.

Using `hrt_call_every()` is the common method used for regular events in drivers where the operations are very fast, such as SPI device drivers. The operations are typically run with interrupts disabled, and should take only a few tens of microseconds at most.

If you compare this to the hmc5883 driver, you will instead see a line like this:

```
work_queue(HPWORK, &_work, (worker_t)&HMC5883::cycle_trampoline, this, 1);
```

that uses an alternative mechanism for regular events which is suitable for slower devices, such as I2C devices. What this does is add the `cycle_trampoline` function to a work queue within the hpwork thread that you saw above. Calls made within HPWORK workers should run with interrupts enabled and may take up to a few hundred microseconds. For tasks which will take longer than that the LPWORK work queue should be used, which runs them in the lower priority lpwork thread.

The AP_Scheduler system

The next aspect of ArduPilot threading and tasks to understand is the AP_Scheduler system. The AP_Scheduler library is used to divide up time within the main vehicle thread, while providing some simple mechanisms to control how much time is used for each operation (called a 'task' in AP_Scheduler).

The way it works is that the loop() function for each vehicle implementation contains some code that does this:

- wait for a new IMU sample to arrive
- call a set of tasks between each IMU sample

It is a table driven scheduler, and each vehicle type has a AP_Scheduler::Task table. To learn how it works have a look at the [AP_Scheduler/examples/Scheduler_test.cpp](#) sketch.

If you look inside that file you will see a small table with a set of 3 scheduling tasks. Associated with each task are two numbers. The table looks like this:

```
static const AP_Scheduler::Task scheduler_tasks[] PROGMEM = {
  { ins_update, 1, 1000 },
  { one_hz_print, 50, 1000 },
  { five_second_call, 250, 1800 },
};
```

The first number after each function name is the call frequency, in units controlled by the ins.init() call. For this example sketch the ins.init() uses RATE_50HZ, so each scheduling step is 20ms. That means the ins_update() call is made every 20ms, the one_hz_print() function is called every 50 times (ie. once a second) and the five_second_call() is called every 250 times (ie. once every 5 seconds).

The second number is the maximum time that the function is expected to take. This is used to avoid making the call unless there is enough time left in this scheduling run to run the function. When scheduler.run() is called it is passed the amount of time (in microseconds) available for running tasks, and if the worst case time for this task would mean it wouldn't fit before that time runs out then it won't be called.

Another point to look at closely is the ins.wait_for_sample() call. That is the "metronome" that drives the scheduling in ArduPilot. It blocks execution of the main vehicle thread until a new IMU sample is available. The time between IMU samples is controlled by the arguments to the ins.init() call.

Note that tasks in AP_Scheduler tables must have the following attributes:

- they should never block (except for the `ins.update()` call)
- they should never call sleep functions when flying (an autopilot, like a real pilot, should never sleep while flying)
- they should have predictable worst case timing

You should now go and modify the `Scheduler_test` example and add in your own tasks to run. Try adding tasks that do the following:

- read the barometer
- read the compass
- read the GPS
- update the AHRS and print the roll/pitch

Look at the example sketches for each library that you worked with earlier in this tutorial to understand how to use each sensor library.

Semaphores

When you have multiple threads (or timer callbacks) you need to ensure that data structures shared by the two logical threads of execution are updated in a way that prevents corruption. There are 3 principle ways of doing this in ArduPilot - semaphores, lockless data structures and the PX4 ORB.

AP_HAL Semaphores are just wrappers around whatever semaphore system is available on the specific platform, and provide a simple mechanism for mutual exclusion. For example, I2C drivers can ask for the I2C bus semaphore to ensure that only one I2C device is used at a time.

Go and have a look at the `hmc5843` driver in `libraries/AP_Compass/AP_Compass_HMC5843.cpp` and look for the `get_semaphore()` call. Look at all the places it is used, and see if you can work out why it is needed.

Lockless Data Structures

The ArduPilot code also contains examples of using lockless data structures to avoid the need for a semaphore. This can be a lot more efficient than semaphores.

Two examples of lockless data structures in ArduPilot are:

- the `_shared_data` structure in `libraries/AP_InertialSensor/AP_InertialSensor_MPU9250.cpp`
- the ring buffers used in numerous places. A good example is `libraries/DataFlash/DataFlash_File.cpp`

Go and have a look at these two examples, and prove to yourself that they are safe for concurrent access. For DataFlash_File look at the use of the `_writebuf_head` and `_writebuf_tail` variables.

It would be nice to create a generic ring buffer class which could be used instead of the separate ringbuffer implementations in several places in ArduPilot. If you want to contribute that then please do a pull request!

The PX4 ORB

Another example of this type of mechanism is the PX4 ORB. The ORB (Object Request Broker) is a way of providing data from one part of the system to another (eg. device driver -> vehicle code) using a publish/subscribe model that is safe in a multi-threaded environment.

The ORB provides a nice mechanism for declaring structures which will be shared in this way (all defined in [PX4Firmware/src/modules/uORB/](#)). Code can then “publish” data to one of these topics, which is picked up by other pieces of code.

An example is the publication of actuator values so the uavcan ESCs can be used on Pixhawk. Have a look at the `_publish_actuators()` function in `AP_HAL_PX4/RCOutput.cpp`. You will see that it advertises a “actuator_direct” topic, which contains the speed desired for each ESC. The uavcan code then watches for changes to this topic in [PX4Firmware/src/modules/uavcan/uavcan_main.cpp](#) and outputs the new values to the uavcan ESCs.

Two other common mechanisms for communicating with PX4 drivers are:

- ioctl calls (see the examples in `AP_HAL_PX4/RCOutput.cpp`)
- `/dev/xxx` read/write calls (see `_timer_tick` in `AP_HAL_PX4/RCOutput.cpp`)

Please talk to the ardupilot development team on the [ArduPilot Developers Discord](#) if you are not sure which mechanism to use for new code.

UARTs and the Console

A lot of components in ArduPilot rely on UARTs. They are used for debug output, telemetry, GPS modules, and more. Understanding how to talk to the UARTs via the HAL will help you understand a lot of ArduPilot code.

The 8 UARTs

The ArduPilot HAL currently defines 8 UARTs. The HAL itself does not define any particular roles for these UARTs, but the other parts of ArduPilot assume they will be assigned particular functions. The command-line options for using with `sim_vehicle.py` the serial port should be preceded by `-A` to pass along to the vehicle binary. Make sure to include the `uart` protocol. Specifying a baudrate is not required, but is more consistent. For example, `sim_vehicle.py --console --map -A --serial5=uart:/dev/ttyS15:115200`.

ParamPrefix	Sim_vehicle Cmd Line	Default Role	Default Connection
SERIAL0_	- -serial0=	Console	tcp:localhost:5760:wait
SERIAL1_	- -serial1=	MAVLink	tcp:localhost:5762
SERIAL2_	- -serial2=	MAVLink	tcp:localhost:5763
SERIAL3_	- -serial3=	GPS	Simulated GPS
SERIAL4_	- -serial4=	GPS	Simulated GPS
SERIAL5_	- -serial5=		
SERIAL6_	- -serial6=		
SERIAL7_	- -serial7=		

The following are options for connecting a SITL serial port:

- Passthrough to a real serial device: `--serialX=uart:<device>:<baudrate>`
- TCP server: `--serialX=tcp:<port>:wait`
- TCP client: `--serialX=tcpclient:<remote IP>:<port>`
- UDP client: `--serialX=udpclient:<remote IP>:<port>`
- UDP multicast: `--serialX=mcast:<multicast IP>:<port>`
- Log to file: `--serialX=file:<path and filename>`
- Simulated sensor: `--serialX=sim:<device name>`

A few examples of usage are:

```
--serial1=uart:/dev/ttyUSB0:57600
--serial1=uart:/dev/ttyS15:115200 #Cygwin comm ports are ttyS and they start at 0, so 15 is equivalent to COM16
--serial1=tcp:5800:wait # wait means pause SITL startup until a connection is established. Otherwise use nowait
--serial1=tcpclient:192.168.1.110:5760
--serial1=udpclient:192.168.1.110:14550
--serial1=mcast:239.255.145.50:14550
--serial1=file:/tmp/my-device-capture.BIN
--serial1=sim:ParticleSensor_SDS021:
```

When using with `sim_vehicle.py`, ensure that the `-A` option is used to pass the configuration option to the SITL executable. For example:

```
sim_vehicle.py -v ArduCopter -A "--serial1=uart:/dev/ttyUSB0:115200" --console --map
```

If you are writing your own sketch using the ArduPilot HAL then you can use these UARTs for any purpose you like, but if possible you should try to use the above assignments as it will allow you to fit in more easily to existing code.

You can change the role of UART by changing its SERIALn_PROTOCOL param. Possible parameter values are listed in the description for [SERIAL1_PROTOCOL](#).

Go and have a look at the [libraries/AP_HAL/examples/UART_test](#) example sketch. It prints a hello message to the 1st 5 UARTs. Try it on your board and see if you can get all the outputs displaying using a USB serial adapter. Try changing the baudrate in the sketch.

Debug console

Historically, In addition to the basic 5 UARTs there was an additional debug console available on some platforms. Recently debug console is directed to USB. On SITL, debug is directed to a terminal, while USB is directed to port 5760 by default.

If you have a board that does have HAL_OS_POSIX_IO set (check that in [AP_HAL/AP_HAL_Boards.h](#)) then try adding some `::printf()` and other stdio functions to the UART_test sketch.

If `::printf` doesn't work for you, it may be that your particular file (e.g. a library) does not have `#include <stdio.h>` at the top of it, just add it. :-)

You can also use `hal.console->printf()` to specify USB port.

UART Functions

Every UART has a number of basic IO functions available. The key functions are:

- `printf` - formatted print
- `printf_P` - formatted print with progmem string (saves memory on AVR boards)
- `println` - print and line feed
- `write` - write a bunch of bytes
- `read` - read some bytes
- `available` - check if any bytes are waiting
- `txspace` - check how much outgoing buffer space is available
- `get_flow_control` - check if the UART has flow control capabilities

Go and have a look at the declarations of each of these in `AP_HAL` and try them in `UART_test`.

RC Input and Output

RC input is a key part of any autopilot, giving the pilot control of the airframe, allowing them to change modes and also giving them control of auxiliary equipment such as camera mounts.

The number of channels available depends on the hardware of the particular board. Note that SBUS and Spektrum/DSM are serial protocols. SBUS is a 100kbaud inverted UART protocol and Spektrum/DSM is a 115200 baud UART protocol. Some boards implement these using hardware UARTs (such as on PX4) and some implement them as bit-banged software UARTs (on Linux).

RC Output is how ArduPilot controls servos and motors. The number of available output channels depends on the type of board, and can even depend on the vehicle type and configuration parameters. RC Output defaults to 50Hz PWM values, but can be configured for a wide range of update rates. For example, the Copter code sets up its motor outputs to drive the ESCs at a much higher rate - typically over 400Hz.

AP_HAL RCInput object

The first object to understand is the AP_HAL RCInput object which is available as `hal.rcin`. That provides low level access to the channel values currently being received on the board. The returned values are PWM values in microseconds.

Go and have a look at the [libraries/AP_HAL/examples/RCInput/RCInput.cpp](#) sketch and try it on your board. Try moving the sticks on your transmitter and check that the values change correctly in the output.

AP_HAL RCOOutput object

The AP_HAL RCOOutput object (available as `hal.rcout`) gives low level control of all output channels. How this is implemented is very board specific, and may involve programming of on-chip timers, or an I2C peripheral, or output via a co-processor (such as the PX4IO microcontroller).

Go and have a look at the [libraries/AP_HAL/examples/RCOutput/RCOutput.cpp](#) example sketch. You'll see that it just sets up all the channels to wave the servos from minimum to maximum over a few second period. Hook up some servos to your board and then test to

make sure it works for you.

The RC_Channel object

The hal.rcin and hal.rcout objects discussed above are low level functions. The usual way of handling RC input and output in ArduPilot is via a higher level object called RC_Channel. That object has user configurable parameters for the min/max/trim of each channel, as well as support for auxiliary channel function, scaling of inputs and outputs and many other features.

Go and have a look at [libraries/RC_Channel/examples/RC_Channel/RC_Channel.cpp](#). That example shows how to setup RC channels, read input and copy input to output values. Run that on your board and check that transmitter input is passed through to a servo. Try changing it to reverse a channel, and change the min/max/trim on a channel. Have a look through RC_Channel.h to see what API functions are available.

The RC_Channel_aux object

Along with RC_Channel there is another important class in libraries/RC_Channel. It is the RC_Channel_aux class, which is a subclass of RC_Channel.

An RC_Channel_aux object is a type of RC_Channel but with additional properties that allow its purpose to be specified by a user. Say for example a user wants channel 6 to be a roll stabilization for a camera mount. They would set a parameter RC6_FUNCTION to be 21, which means “rudder”. Then another library could say:

```
RC_Channel_aux::set_servo_out(RC_Channel_aux::k_rudder, 4500);
```

and any channel which has its FUNCTION set to 21 will move to full deflection (as k_rudder is setup scaled as an angle in centi-degrees with 4500 being the maximum). Note that this is a one to many arrangement. The user can setup multiple channels to have FUNCTION 21 if they want to, and all of those channels will move, each using their own min/max/trim values.

Storage and EEPROM management

Every board that ArduPilot supports has some form of persistent storage available. This is used to hold user parameters, waypoints, rally points, terrain data and many other useful things. To provide access to this storage ArduPilot has 4 basic mechanisms:

- the `AP_HAL::Storage` object, accessed as `hal.storage`
- the `StorageManager` library to give a higher level abstraction layer on `hal.storage`
- `DataFlash` for storing to an on-board logging area
- Posix IO functions to traditional filesystems (for example VFAT on a microSD card), on boards that support it

The other libraries and functions that need persistent storage are built on these basic systems. For example, the `AP_Param` library (which handles user settable parameters) is built on top of `StorageManager`, which in turn is built on top of `AP_HAL::Storage`. The `AP_Terrain` library (which handles terrain data) is built on top of Posix IO functions for holding the terrain database.

The `AP_HAL::Storage` library

The `AP_HAL::Storage` object is available on all platforms. The minimum size of storage available via this interface on boards that ArduPilot supports is 4096 bytes. Some boards provide more space - for example the PX4v1 has 8k of EEPROM and the Pixhawk has 16k of FRAM. All of that is hidden behind the `AP_HAL::Storage` API.

The `hal.storage` API is extremely simple. It has just 3 functions

- `init()` to start up the storage subsystem
- `read_block()` to read a block of bytes
- `write_block()` to write a block of bytes

The reason for this very simple API is that developers are encouraged to use the `StorageManager` API instead of `hal.storage`. You should only be delving into `hal.storage` when doing bringup of a new board, or when debugging.

The size of storage available is set inside [AP_HAL/AP_HAL_Boards.h](#) in the macro `HAL_STORAGE_SIZE`. This means we don't yet support dynamically determining the size of storage available for this interface. If you want dynamically sized storage you need to use

Posix IO for now.

We don't have an example sketch for the AP_HAL::Storage API, so this is your chance to write one. If you have gotten this far into the ArduPilot tutorial you should have seen enough example sketches to know how to write one from scratch. So write a `libraries/AP_HAL/examples/Storage` example that calculates an 8 bit XOR of the full contents of the `hal.storage` data, and prints it to the console. Then [submit the example as a patch](#) to the ArduPilot github, being careful to follow the patch submission guidelines.

I will be quite interested to see how long after I add this exercise to the tutorial before we get a submission

The StorageManager library

The simple `hal.storage` API makes it nice and easy to port ArduPilot to a new board, but isn't convenient for actually using the available storage. For that we have `StorageManager`. The `StorageManager` library provides an API which abstracts the storage into pseudo-contiguous blocks of data that have an assigned purpose. So we chop up the available storage into areas that provide:

- parameters
- fence points
- waypoints
- rally points
- signing key
- RC bind info

Go and have a read of [libraries/StorageManager/StorageManager.cpp](#). In particular look at the tables at the top. Notice how multiple areas of each type are defined for systems with have larger amounts of storage. This ability to combine multiple non-contiguous areas of storage into a single logical storage area was the main motivation for adding `StorageManager`. It allowed us to grow from using 4k of storage on all boards to using the full storage available on each board without asking users to reload all their parameters or reload their waypoints.

This theme of trying to avoid making users reconfigure their autopilot boards is a common one in ArduPilot. We like it when users can update to the latest firmware and everything they previously setup still works, while gaining new features. Sometimes that means we have to do a bit more work as developers to make that possible - such as happened with `StorageManager`.

The `StorageManager` API also provides convenient functions for reading and writing variables like integers. This is the API that libraries like `AP_Mission` use to save and restore waypoints.

Now go and have a look at [libraries/StorageManager/examples/StorageTest.cpp](#). This is a stress test for the StorageManager layer, and as a result is also a stress test for the AP_HAL::Storage object. It does random IO at random offsets of random lengths. That means it does IOs that cross boundaries where a single parameter value may be non-contiguous in FRAM or EEPROM. This test sketch was designed to stress test the StorageManager API, but it is also terribly useful when porting ArduPilot to a new board, as it stresses the EEPROM access functions very nicely.

Go and try StorageTest on your board, but be warned that it is a destructive test. It won't destroy your board, but it will wipe all your parameters and waypoints. So backup your configuration if you are testing on the board in your favourite aircraft.

As an exercise, add some profiling to StorageTest so it prints the total IO rate in kbytes/sec along with the IO rate for reads and writes. Do you notice something about the difference between the read and write rates? Do you notice anything about the speed of writing values which are already set at that address in storage? See if you can find the code in StorageManager that explains your observations. Then [submit a patch](#) adding the profiling output to the ArduPilot github.

Next go and search through the ArduPilot libraries to find all the places that use the StorageManager. What you are looking for is StorageAccess handles, like this:

```
StorageAccess AP_Param::_storage(StorageManager::StorageParam);
```

that declares a variable called AP_Param::_storage which provides access to the StorageParam area of the storage on this board. What libraries use StorageAccess?

The DataFlash library

Another type of storage that autopilots need is for on-board logs. For ArduPilot that is provided by the DataFlash library. It is quite an odd library in many ways. For a start the name is weird and comes from the fact that it was originally designed around the DataFlash chip for the APM1. It was a hardware device driver library which morphed over time into a general logging system. Internally the structure of the library shows this history in several ways (and not good ways!).

These days the DataFlash API is designed around a logging infrastructure model. It allows you to define self-describing data structures for individual log messages - such as a "GPS" message to log data from a GPS sensor. It handles logging that data to persistent storage in an efficient manner and also provides APIs for other libraries to use to get the data back out when the user wants to download their log files after a flight.

If you have seen the ‘*.bin’ files that ArduPilot uses these days when you download a log then you have seen the format that ArduPilot uses to store log messages. It is “self describing”, meaning that the ground station can work out the format of the messages in the log file without having to have some common scheme. At the front of each log file is a set of FMT messages which have a well known format and which describe the format of the messages that follow.

Go and have a look at [libraries/AP_FlashStorage/examples/FlashTest/FlashTest.cpp](#). You’ll see a little table at the top that defines the log messages we will be writing, in this case a ‘TEST’ message which contains 4 unsigned 16 bit integers and two signed 32 bit integers (that is what “HHHHii” means). It also gives names for those 6 variables (cunningly labelled V1 to V4 and L1 and L2).

In the loop() function you will see a rather strange call like this:

```
DataFlash.get_log_boundaries(log_num, start, end);
```

This is the public API for the way that the DataFlash library hides how the board actually stores log files. On a system that has Posix IO (such as Pixhawk or Linux) log files are stored as separate files in a “LOGS” directory on the microSD card. These files can be directly copied by a user by pulling out the microSD card and putting it into their PC.

On a board like the APM2 things aren’t quite that simple. The APM2 has 4 megabytes of storage on a DataFlash chip, accessible across an SPI interface. The interface itself is page oriented, so you need to fill one 512 byte (or possibly 528 byte!) page, then tell the chip to copy that page to persistent storage while you fill the next page. Doing random IO on this DataFlash is not good - it is designed for use by code that needs to write continuously, which is what happens when logging. The 4 megabyte size is really not very large compared to the amount of data an autopilot likes to log, so we need to handle wrapping when it fills up as well.

All of that complexity is hidden behind an API that presents the concept of a “log number”, which is just a bunch of bytes that were written in one flight of the autopilot. The DataFlash implementation on APM1 and APM2 uses little marker bytes at the front of each page to say which log number is being written. These log numbers correspond to the log numbers that are downloaded when the user asks to retrieve their logs.

Posix IO

Some of the autopilot boards that ArduPilot supports are based on an operating system that has a Posix-like API. For example the Linux ports have a very good Posix subsystems, and the NuttX operating system used for PX4 (such as on Pixhawk) have a pretty reasonable Posix layer. You can take advantage of this in libraries for ArduPilot as long as you don’t rely on it for anything that has to work on all boards.

A good example of this is the `AP_Terrain` library, which holds terrain data. Terrain data is much too large to fit in EEPROM, and it is random access, so it isn't suitable for DataFlash. It is also not essential for the basic functionality of an autopilot, so it is a great candidate for implementing using Posix IO.

The way we use Posix IO is that you first check if the board has Posix IO support by checking the `HAVE_OS_POSIX_IO` macro from [AP_HAL_Boards.h](#). Then to know where on the filesystem you should store the data you add a data specific macro in `AP_HAL_Boards.h` which gives the directory path where that sort of data should be placed. For example, the macro `HAL_BOARD_TERRAIN_DIRECTORY` is used to define the directory where terrain data should go.

Once you have those two things you should just use normal Posix IO functions (ie. open, close, read, write etc), although there are some caveats:

- you must only call IO functions from either the IO timer or from your own low priority thread.
- Never call any IO functions from APIs directly accessible in your library. Not even a simple one like `stat()`.
- try to be friendly to slow storage cards, do IO in reasonably sized chunks, and avoid seeks where possible

These rules really matter. A simple IO on a microSD card on Pixhawk could take up to a second. That is enough time for your precious quadcopter to be flying upside down and heading for its final meeting with the ground. The average time for an IO on a Pixhawk microSD card is quite low (a few milliseconds), but just occasionally you will get a slow one when the microSD card decides it need to spend some quality time re-reading the SD card specification and calculating Pi. Don't be tempted to sneak a little operation in just because it seems fast most of the time.

The one exception to this is initialization functions that you know for certain can only be called when the vehicle is starting up or is disarmed. A bit of delay at that time is fine.

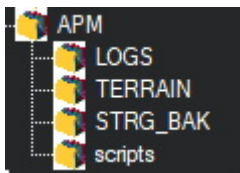
Now go and have a read of [libraries/AP_Terrain/TerrainIO.cpp](#) and look at how it uses Posix IO. Notice the little state machine it uses to handle all the IO, which is all called from the `AP_Terrain::io_timer` function. See if you can spot any bugs, and report them if you can!

ArduPilot File Systems

ArduPilot uses several files systems to store and retrieve data from non-volatile storage using either external SD cards or autopilot flash memory.

External SD Card

The directory structure is:



- APM: base directory
- LOGS : flight logs are stored here (See [Downloading and Analyzing Data Logs in Mission Planner](#))
- TERRAIN : terrain data is stored here (See [Terrain Following](#))
- STRNG_BAK: every boot the parameter data is backed up here
- scripts: LUA scripts are stored here (See [Lua Scripts](#))

and a vehicle backup file (one for each vehicle class that the card has been used) that contains ???

Flash based virtual file systems

Several flash based file systems are setup. Most are for internal, ground station, or developer use only, but one is useful for normal users.

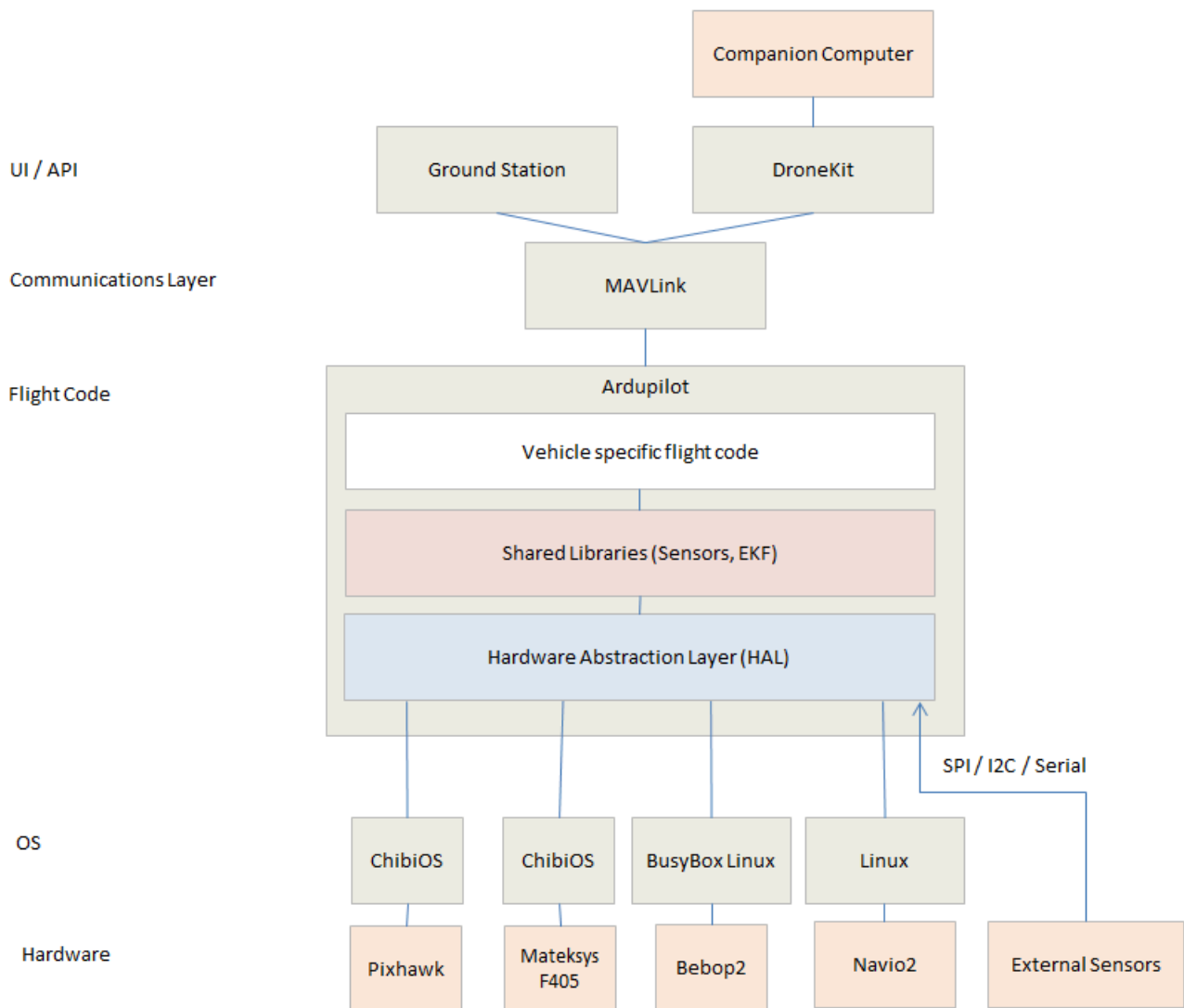
- @ROMFS: this contains a copy of the bootloader version created with the firmware that is loaded which is used if a bootloader update is commanded from an attached ground station or pc utility. It also contains a copy of the hardware definition file, and font files if OSD support is included.
- @PARAM: this is a packed version of the firmware's parameters which can be downloaded from the ground control station.
- @SYS: which contains several text files that keep statistics of internal operational processes for information and debugging issues (really the only one a normal user may be asked to access to send information when debugging a user issue):

- threads.text: information on the concurrent process threads during operation
- tasks.text:
- dma.text:
- memory.txt:
- uarts.txt:

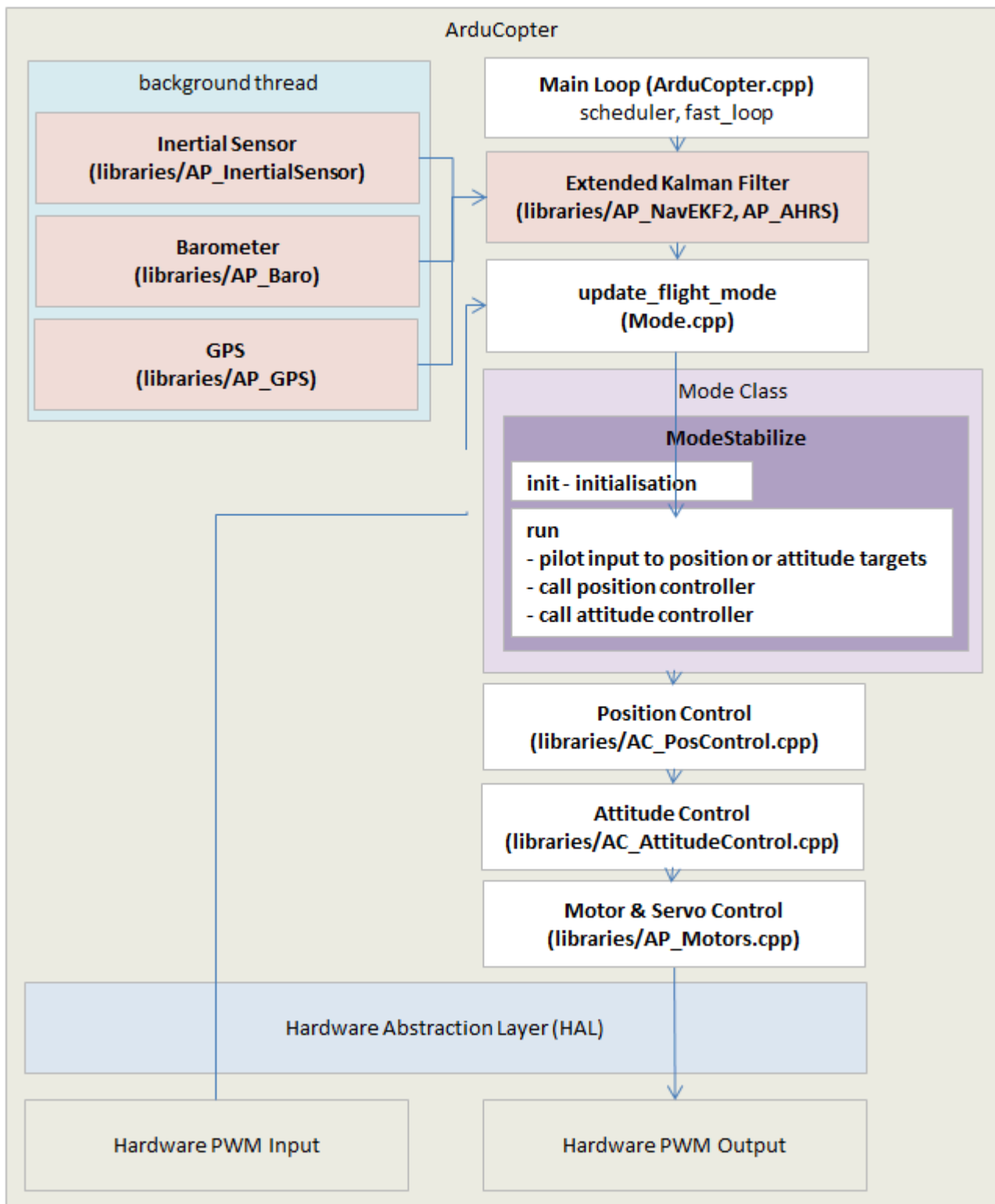
Code Overview (Copter)

The **code** is made up of [the main Copter code](#) which resides in its own directory, and [the libraries](#) which are shared with Plane and Rover.

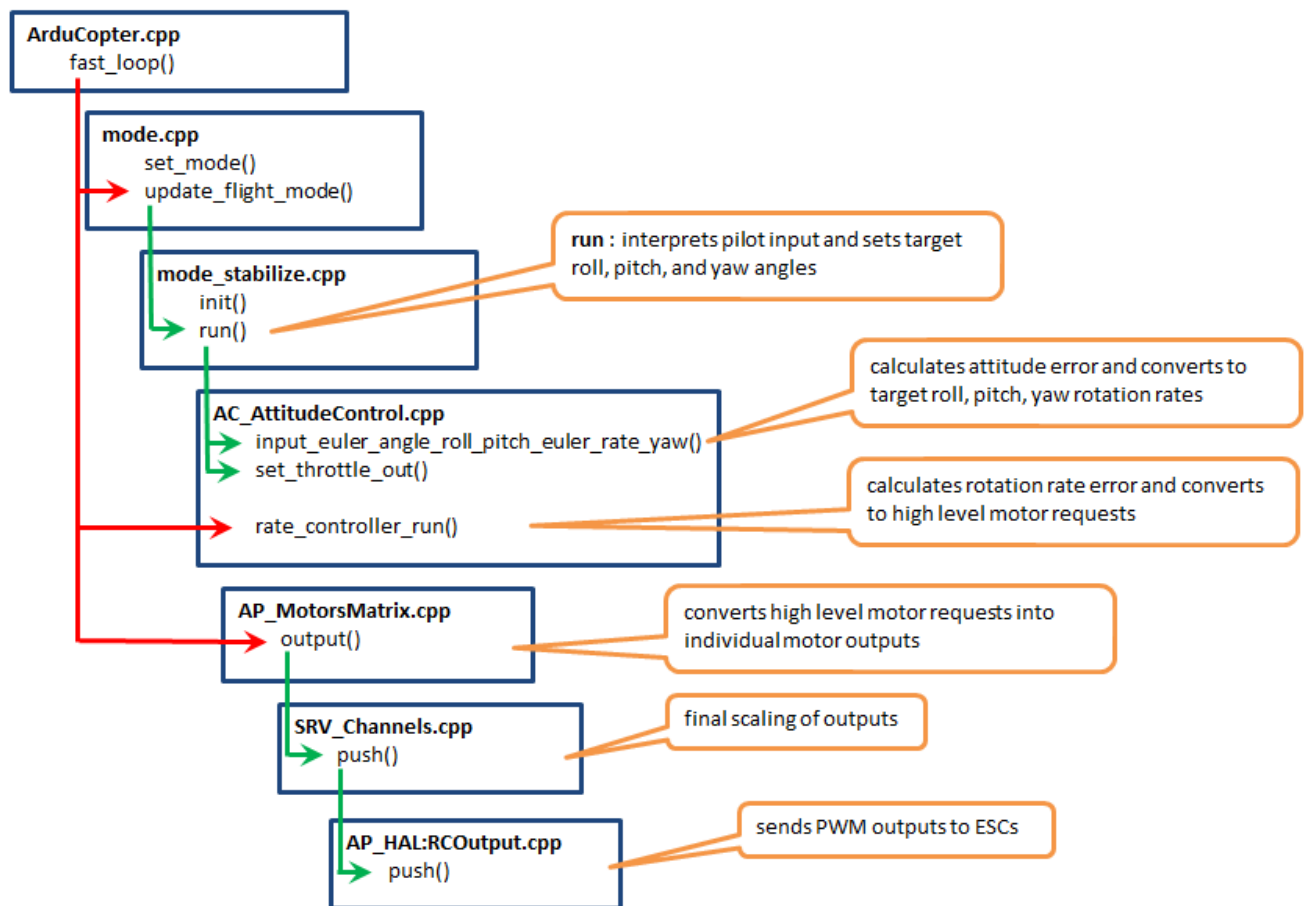
Below is a highlevel view of the ardupilot architecture.



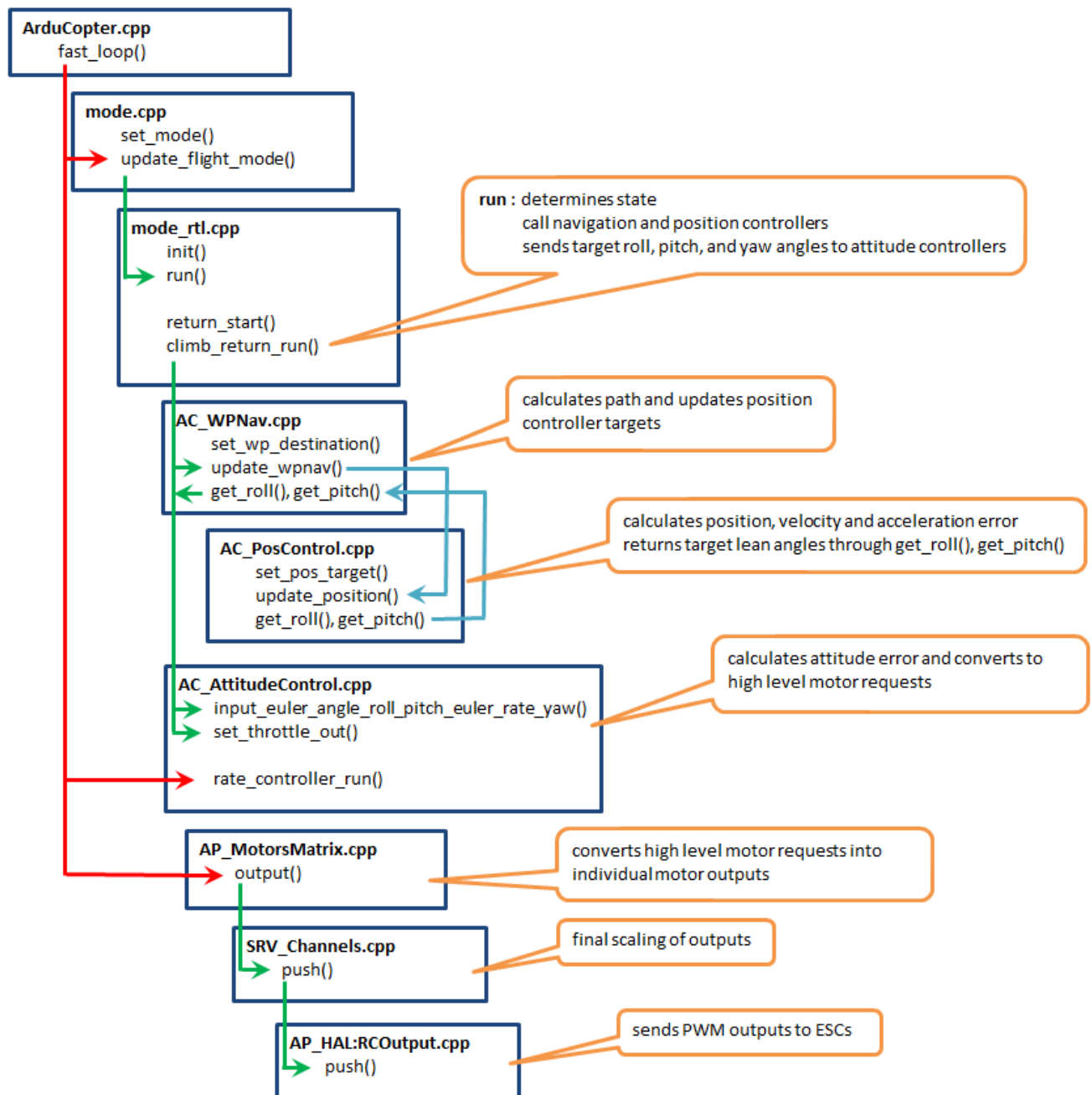
Below is a more zoomed in view (as compared to the above diagram) of the architecture.



The image below shows the architecture of **manual modes** (i.e. Stabilize, Acro, Drift)

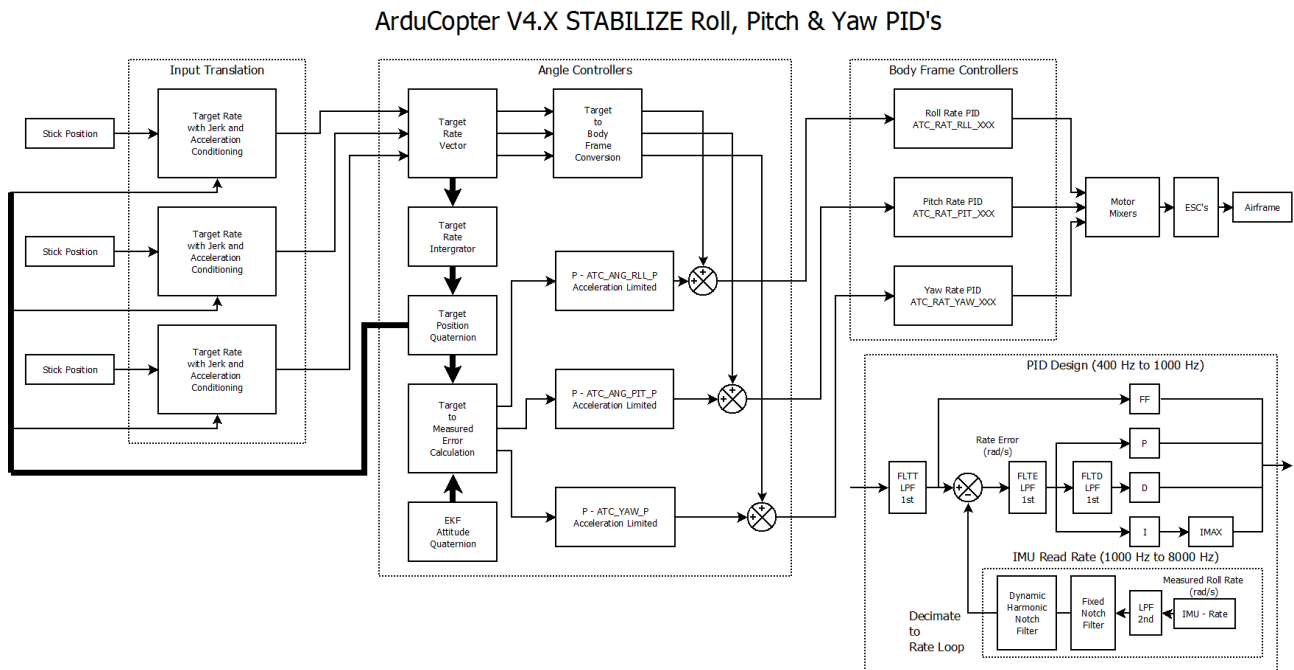


The image below shows the architecture of **autonomous modes** (i.e. RTL, Guided, Auto)

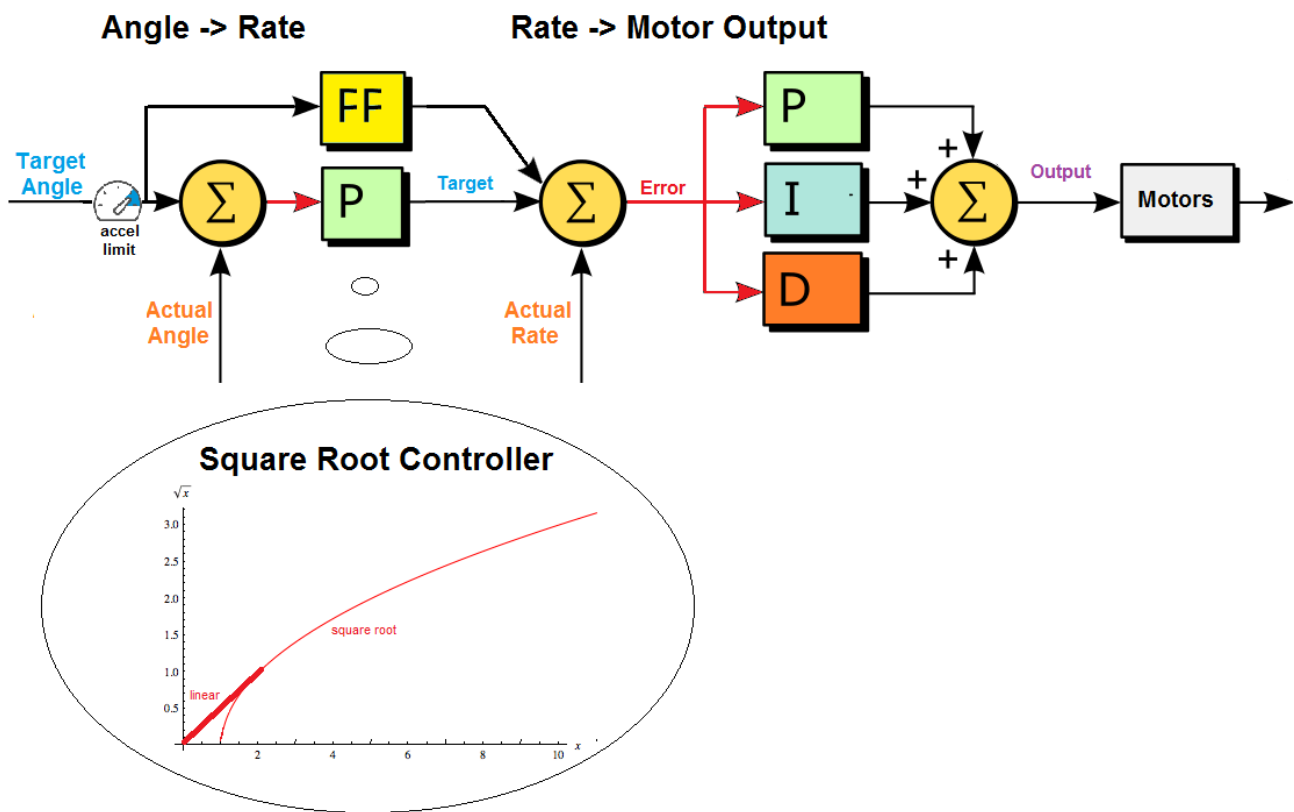


Copter Attitude Control

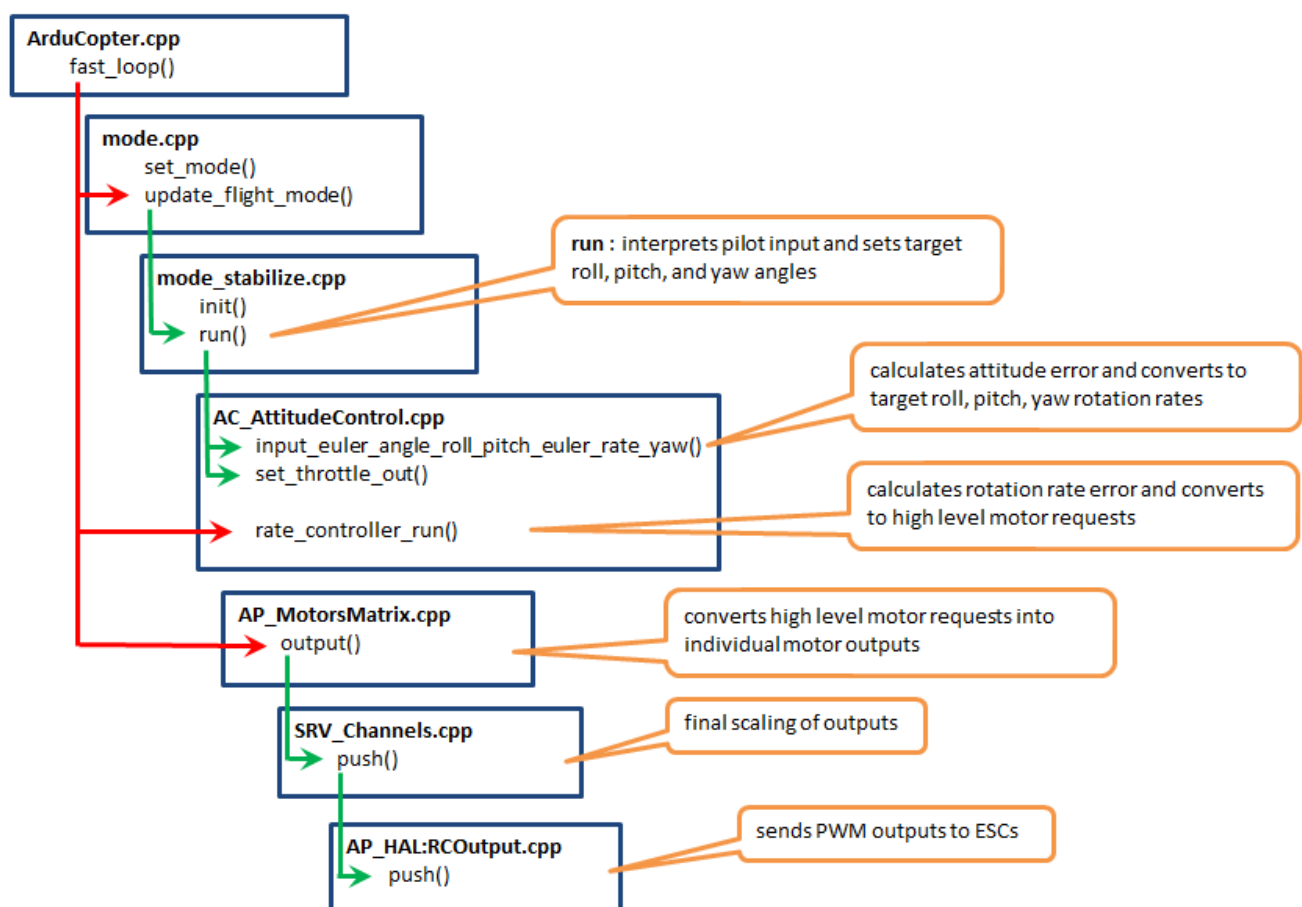
Below is a high level diagram showing how the attitude control is done.



Below describes what is done for each axis. A P controller converts the angle error (the difference between the target angle and actual angle) into a desired rotation rate followed by a PID controller to convert the rotate rate error into a high level motor command. The “square root controller” portion of the diagram shows the curve used with the angle control's P controller.



The diagram below shows the code path followed from pilot input down to pwm output.



On every update (i.e. 400hz on Pixhawk, 100hz on APM2.x) the following happens:

- the top level flight-mode.cpp's "update_flight_mode()" function is called. This function checks the vehicle's flight mode (ie. "control_mode" variable) and then calls the appropriate <flight mode>_run() function (i.e. [stabilize_run](#) for stabilize mode, [rtl_run](#) for

RTL mode, etc). The `<flight mode>_run()` function can be found in the appropriately named `mode_<flight mode>.cpp` file (i.e. [mode_stabilize.cpp](#), [mode_rtl.cpp](#), etc).

- the `<flight mode>_run` function is responsible for converting the user's input (found in `g.rc_1.control_in`, `g.rc_2.control_in`, etc) into a lean angle, rotation rate, climb rate, etc that is appropriate for this flight mode. For example [AltHold](#) converts the user's roll and pitch input into lean angles (in degrees), the yaw input is converted into a rotation rate (in degrees per second) and the throttle input is converted to a climb rate (in cm/s).
- the last thing the `<flight mode>_run` function must do is pass these desired angles, rates etc into Attitude Control and/or Position Control libraries (these are both held in the [AC_AttitudeControl](#) folder).
- The [AC_AttitudeControl](#) library provides 5 possible ways to control the attitude of the vehicle, the most common 3 are described below.
 - [angle_ef_roll_pitch_rate_ef_yaw\(\)](#) : this accepts an "earth frame" angle for roll and pitch, and an "earth frame" rate for yaw. For example providing this function `roll = -1000`, `pitch = -1500`, `yaw = 500` means lean the vehicle left to 10degrees, pitch forward to 15degrees and rotate right at 5deg/second.
 - [angle_ef_roll_pitch_yaw\(\)](#) : this accepts "earth frame" angles for roll, pitch and yaw. similar to above except providing yaw of 500 means rotate the vehicle to 5 degrees east of north.
 - [rate_bf_roll_pitch_yaw\(\)](#) : this accepts a "body frame" rate (in degrees/sec) for roll pitch and yaw. For example providing this function `roll = -1000`, `pitch = -1500`, `yaw = 500` would lead to the vehicle rolling left at 10deg/sec, pitching forward at 15deg/sec and rotating about the vehicle's z axis at 5 deg/sec.

After any calls to these functions are made the [AC_AttitudeControl::rate_controller_run\(\)](#) is called. This converts the output from the methods listed above into roll, pitch and yaw inputs which are sent to the [AP_Motors](#) library via it's [set_roll](#), [set_pitch](#), [set_yaw](#) and [set_throttle](#) methods.

- The [AC_PosControl](#) library allows 3D position control of the vehicle. Normally only the simpler Z-axis (i.e. altitude control) methods are used because more complicated 3D position flight modes (i.e. [Loiter](#)) make use of the [AC_WPNV](#) library. In any case, some commonly used methods of this library include:
 - [set_alt_target_from_climb_rate\(\)](#) : this accepts a climb rate in cm/s and updates an absolute altitude target
 - [set_pos_target\(\)](#) : this accepts a 3D position vector which is an offset from home in cm

If any methods in [AC_PosControl](#) are called then the flight mode code must also call the [AC_PosControl::update_z_controller\(\)](#) method. This will run the z-axis position control PID loops and send low-level throttle level to the [AP_Motors](#) library. If any xy-axis methods are called then [AC_PosControl::update_xy_controller\(\)](#) must be called.

- The [AP_Motors](#) library holds the "motor mixing" code. This code is responsible for converting the roll, pitch, yaw and throttle values received from the [AC_AttitudeControl](#) and [AC_PosControl](#) libraries into absolute motor outputs (i.e. PWM values). So the higher

level libs would make use of these functions:

- `set_roll()`, `set_pitch()`, `set_yaw()` : accepts roll, pitch and yaw values in the range of -4500 ~ 4500. These are not desired angles or even rates but rather just a value. For example `set_roll(-4500)` would mean roll left as fast as possible.
- `set_throttle()` : accepts an absolute throttle value in the range of 0 ~ 1000. 0 = motors off, 1000 = full throttle.
- There are different classes for each frame type (quad, Y6, traditional helicopter) but in each there is an “`output_armed`” function which is responsible for implementing the conversion of these roll, pitch, yaw and throttle values into pwm outputs. This conversion often includes implementing a “stability patch” which handles prioritising one axis of control over another when the input requests are outside the physical limits of the frame (i.e. max throttle and max roll is not possible with a quad because some motors must be less than others to cause a roll). At the bottom of the “`output_armed`” function there is a call to the `hal.rcout->write()` which passes the desired pwm values to the AP_HAL layer.
- The `AP_HAL` libraries (hardware abstraction layer) provides a consistent interface for all boards. In particular the `hal.rc_out_write()` function will cause the specified PWM received from the AP_Motors class to appear on the appropriate pwm pin out for the board.

Leonard Hall, Developer Un-Conference 2018

Practical PID implementation and the new Position Controller



Adding a New Parameter to Copter

Parameters can either be part of the main code or part of a library.

Adding a parameter to the main code

Step #1: Find a spare slot in the Parameters class's enum in [Parameters.h](#) and add your new parameter as shown below in red. Some things to be careful of:

- Try to add the parameter close to parameters that share a similar function or worst case add to the end of the "Misc" section.
- Ensure that the section you are adding it to is not full. You can check if the section is full by counting the number of parameters in the section and ensuring that the last element is less than the next sections first element. I.e. the Misc section's first parameter is #20. my_new_parameter is #36. If the next section began at #36 we would not be able to add the new parameter here.
- Do not add a parameter in the middle of a group thus causing the slots for existing parameters to change
- Do not use slots with "deprecated" or "remove" comments beside them because some users may still have the defaults for these old parameters in their eeprom so your new parameter's default value could be set strangely.

```

enum {
    // Misc
    //
    k_param_log_bitmask = 20,
    k_param_log_last_filename,          // *** Deprecated - remove
                                        // with next eeprom number
                                        // change
    k_param_toy_yaw_rate,                // THOR The memory
                                        // location for the
                                        // Yaw Rate 1 = fast,
                                        // 2 = med, 3 = slow

    k_param_crosstrack_min_distance,    // deprecated - remove with next eeprom number change
    k_param_rssi_pin,
    k_param_throttle_accel_enabled,     // deprecated - remove
    k_param_wp_yaw_behavior,
    k_param_acro_trainer,
    k_param_pilot_velocity_z_max,
    k_param_circle_rate,
    k_param_sonar_gain,
    k_param_ch8_option,
    k_param_arming_check_enabled,
    k_param_sprayer,
    k_param_angle_max,
    k_param_gps_hdop_good,              // 35
    k_param_my_new_parameter,           // 36
}

```

Step #2: Declare the variable within the Parameters class somewhere below the enum. Possible types include AP_Int8, AP_Int16, AP_Float, AP_Int32 and AP_Vector3 (Note: unsigned integers are not currently supported). The name of the variable should be the same as the new enum but with the “k_param_” prefix removed.

```

    // 254,255: reserved
};

AP_Int16      format_version;
AP_Int8       software_type;

// Telemetry control
//
AP_Int16      sysid_this_mav;
AP_Int16      sysid_my_gcs;
AP_Int8       serial3_baud;
AP_Int8       telem_delay;

AP_Int16      rtl_altitude;
AP_Int8       sonar_enabled;
AP_Int8       sonar_type;      // 0 = XL, 1 = LV,
                                // 2 = XLL (XL with 10m range)
                                // 3 = HRLV

AP_Float      sonar_gain;
AP_Int8       battery_monitoring;    // 0=disabled, 3=voltage only,
                                      // 4=voltage and current

AP_Float      volt_div_ratio;
AP_Float      curr_amp_per_volt;
AP_Int16      pack_capacity;        // Battery pack capacity less reserve
AP_Int8       failsafe_battery_enabled; // battery failsafe enabled
AP_Int8       failsafe_gps_enabled;  // gps failsafe enabled
AP_Int8       failsafe_gcs;         // ground station failsafe behavior
AP_Int16      gps_hdop_good;        // GPS Hdop value below which represent a good
position
AP_Int16      my_new_parameter;    // my new parameter's description goes here

```

Step #3: Add the variable declaration to the var_info table in [Parameters.cpp](#)

```

// @Param: MY_NEW_PARAMETER
// @DisplayName: My New Parameter
// @Description: A description of my new parameter goes here
// @Range: -32768 32767
// @User: Advanced
GSCALAR(my_new_parameter, "MY_NEW_PARAMETER", MY_NEW_PARAMETER_DEFAULT),

```

The @Param ~ @User comments are used by the ground station (i.e. Mission Planner) to display advice to the user and limit the values that the user may set the parameter to.

The parameter name (i.e. "MY_NEW_PARAMETER") is limited to 16 characters.

Step #4: Add the parameters default to [config.h](#)

```

#ifndef MY_NEW_PARAMETER_DEFAULT
# define MY_NEW_PARAMETER_DEFAULT    100    // default value for my new parameter
#endif

```

You're done! The new parameter can be access from the main code (not the libraries) as `g.my_new_parameter`.

Note

ArduPilot vehicle's global parameter classes are crowded. To avoid exceeding the maximum number of parameters in a vehicle's Parameter class, another class for global parameters (named 'ParametersG2') has been introduced. Adding parameters to this class is similar to adding parameters in libraries. The parameters added to this class can similarly be accessed from within the vehicle's main code.

Adding a parameter to a library

Parameters can also be added to libraries by following these steps which use the [AP_Compass](#) library as an example.

Step #1: Add the new class variable to the top level .h file (i.e. [Compass.h](#)). Possible types include `AP_Int8`, `AP_Int16`, `AP_Float`, `AP_Int32` and `AP_Vector3f`. Also add the default value you'd like for the parameter (we will use this in step #2)

```
#define MY_NEW_PARAM_DEFAULT      100

class Compass
{
public:
    int16_t product_id;                ///< product id
    int16_t mag_x;                     ///< magnetic field strength along the X axis
    int16_t mag_y;                     ///< magnetic field strength along the Y axis
    int16_t mag_z;                     ///< magnetic field strength along the Z axis
    uint32_t last_update;              ///< micros() time of last update
    bool healthy;                      ///< true if last read OK

    /// Constructor
    ///
    Compass();

protected:
    AP_Int8 _orientation;
    AP_Vector3f _offset;
    AP_Float _declination;
    AP_Int8 _use_for_yaw;               ///
```

Step #2:Add the variable to the var_info table in the .cpp file (i.e. [Compass.cpp](#)) including `@Param ~ @Increment` comments to allow the GCS to display the description to the user and to control the min and max values set from the ground station. When adding the new parameter be careful that:

- The slot (i.e. “9” below) is one higher than the previous variable.
- the parameter’s name (i.e. MY_NEW_P) length is less than 16 including the object’s prefix that will be added. I.e. the compass object’s prefix is “COMPASS_”.

```
const AP_Param::GroupInfo Compass::var_info[] = {
    // index 0 was used for the old orientation matrix

    // @Param: OFS_X
    // @DisplayName: Compass offsets on the X axis
    // @Description: Offset to be added to the compass x-axis values to compensate for metal
in the frame
    // @Range: -400 400
    // @Increment: 1

<snip>

    // @Param: ORIENT
    // @DisplayName: Compass orientation
    // @Description: The orientation of the compass relative to the autopilot board.
    // @Values: 0:None,1:Yaw45,2:Yaw90,3:Yaw135,4:Yaw180,5:Yaw225,6:Yaw270,7:Yaw315,8:Roll180
    AP_GROUPINFO("ORIENT", 8, Compass, _orientation, ROTATION_NONE),

    // @Param: MY_NEW_P
    // @DisplayName: My New Library Parameter
    // @Description: The new library parameter description goes here
    // @Range: -32768 32767
    // @User: Advanced
    AP_GROUPINFO("MY_NEW_P", 9, Compass, _my_new_lib_parameter, MY_NEW_PARAM_DEFAULT),

    AP_GROUPEND
};
```

The parameter can be accessed from within the library as `_my_new_lib_parameter`. Note that we made the parameter “protected” so it cannot be access from outside the class. If we’d made it public then it would have been accessible to the main code as well as “compass._my_new_lib_parameter”.

Step #3: Add a declaration for `var_info` to the public section of the `.h` file of new library class in addition to the definition in the `.cpp` file:

```
static const struct AP_Param::GroupInfo var_info[];
```

Step #4: If the class is a completely new addition to the code (as opposed to an existing class like `AP_Compass`), it should be added to the main vehicle’s `var_info` table in the [Parameters.cpp](#) file (its order in the `var_info` table is not important). Below in red where the Compass class appears.

```

const AP_Param::Info var_info[] = {
    // @Param: FORMAT_VERSION
    // @DisplayName: Eeprom format version number
    // @Description: This value is incremented when changes are made to the eeprom format
    // @User: Advanced
    GSCALAR(format_version, "FORMAT_VERSION", 0),
<snip>

    // @Group: COMPASS_
    // @Path: ../libraries/AP_Compass/Compass.cpp
    GOBJECT(compass, "COMPASS_", Compass),

<snip>
    // @Group: INS_
    // @Path: ../libraries/AP_InertialSensor/AP_InertialSensor.cpp
    GOBJECT(ins, "INS_", AP_InertialSensor),

    AP_VAREND
};

```

Step #5: If the class is a completely new addition to the code, also add `k_param_my_new_lib` to the enum in [Parameters.h](#), where `my_new_lib` is the first argument to the `GOBJECT` declaration in [Parameters.cpp](#). Read the comments above the enum to understand where to place the new value, as order is important here.

Changing the type of a parameter

From time to time parameters need to be altered or renamed. ArduPilot has capabilities that allow this to be managed from release to release so that upgrades always preserve user configured parameters. Parameters are keyed off slots in the eeprom, so if the occupied slot does not change then the parameter does not change - regardless of its name. If however the type needs to change then the parameter needs to be moved to a new slot and the existing slot be reserved to prevent unexpected configuration. In order for the configuration to be preserved it needs to be copied and converted from the old slot to the new slot.

Step #1: Change the index of the existing parameter to an unused index and add a comment to the effect that the old slot is reserved.

Step #2: In order to figure out the keys for conversion:

- Set the `AP_PARAM_KEY_DUMP` definition to “1” [here in AP_Param.h](#)
- Start the old code in SITL and all the parameter names and their magic numbers will be displayed
- Copy-paste from Copter’s `Parameters.cpp` file’s [existing parameter conversion tables like the ones done for the attitude controller gains](#).

Parameter Conversion Expiration

Parameter conversions are necessary to ensure continuity for users as they upgrade to newer versions of ArduPilot. However, the conversion process does utilise some resources on boot and does add to the flash cost of the code. Therefore we don't keep the conversion code forever more. To make it easier to find and remove the conversions, please add the following as a comment above any parameter conversion that you add:

```
// PARAMETER_CONVERSION - Added: Aug-2021
```

Please use the date format shown above to avoid ambiguity. Please also keep the date on the same line as the key word "PARAMETER_CONVERSION" so that the date can be seen in the return, when grep-ing/searching the code. When adding to an already existing AP_Param::ConversionInfo table, please add the conversions as a new block with the conversion comment above the new block. For example:

```
const AP_Param::ConversionInfo q_conversion_table[] = {
    // PARAMETER_CONVERSION - Added: Jan-2018
    { Parameters::k_param_quadplane, 4044, AP_PARAM_FLOAT, "Q_P_POSZ_P" },      // Q_PZ_P
    { Parameters::k_param_quadplane, 4045, AP_PARAM_FLOAT, "Q_P_POSXY_P" },    // Q_PXY_P
    { Parameters::k_param_quadplane, 4046, AP_PARAM_FLOAT, "Q_P_VELXY_P" },    // Q_VXY_P
    { Parameters::k_param_quadplane, 78,   AP_PARAM_FLOAT, "Q_P_VELXY_I" },    // Q_VXY_I
    { Parameters::k_param_quadplane, 142,  AP_PARAM_FLOAT, "Q_P_VELXY_IMAX" }, // Q_VXY_IMAX

    // PARAMETER_CONVERSION - Added: Aug-2021
    { Parameters::k_param_quadplane, 206,  AP_PARAM_FLOAT, "Q_P_VELXY_FLTE" }, //
Q_VXY_FILT_HZ

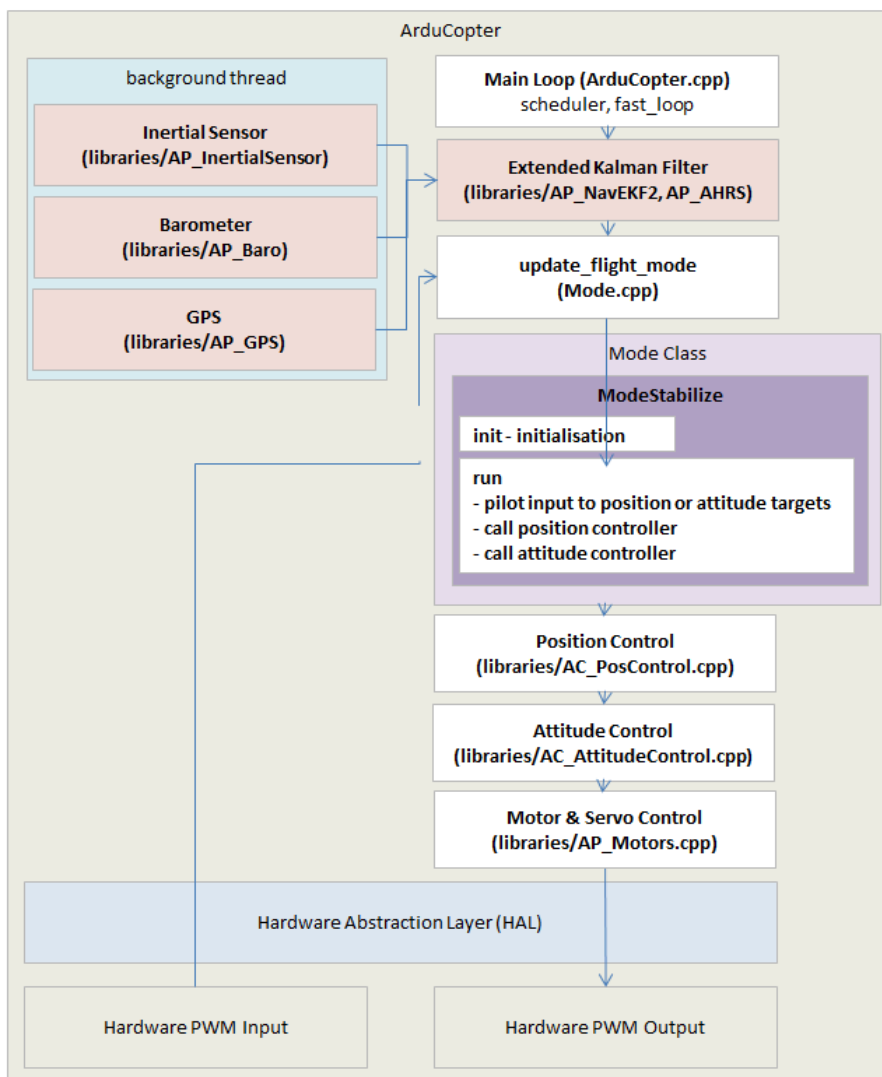
    // PARAMETER_CONVERSION - Added: Jan-2018
    { Parameters::k_param_quadplane, 4047, AP_PARAM_FLOAT, "Q_P_VELZ_P" },      // Q_VZ_P
    { Parameters::k_param_quadplane, 4048, AP_PARAM_FLOAT, "Q_P_ACCZ_P" },      // Q_AZ_P
    { Parameters::k_param_quadplane, 80,   AP_PARAM_FLOAT, "Q_P_ACCZ_I" },      // Q_AZ_I
    { Parameters::k_param_quadplane, 144,  AP_PARAM_FLOAT, "Q_P_ACCZ_D" },      // Q_AZ_D
    { Parameters::k_param_quadplane, 336,  AP_PARAM_FLOAT, "Q_P_ACCZ_IMAX" },    // Q_AZ_IMAX

    // PARAMETER_CONVERSION - Added: Jun-2019
    { Parameters::k_param_quadplane, 400,  AP_PARAM_FLOAT, "Q_P_ACCZ_FLTD" },    // Q_AZ_FILT
```

Adding a New Flight Mode to Copter

This section covers the basics of how to create a new high level flight mode (i.e. equivalent of Stabilize, Loiter, etc)

As a reference the diagram below provides a high level view of Copter's architecture.



1. Pick a name for the new mode and add it to the bottom of the control_mode_t enum in [mode.h](#) just like "NEW_MODE" has been added below.

```
// Auto Pilot Modes enumeration
enum class Number {
    STABILIZE = 0, // manual airframe angle with manual throttle
    ACRO = 1, // manual body-frame angular rate with manual throttle
    ALT_HOLD = 2, // manual airframe angle with automatic throttle
    AUTO = 3, // fully automatic waypoint control using mission commands
    GUIDED = 4, // fully automatic fly to coordinate or fly at velocity/direction
using GCS immediate commands
    LOITER = 5, // automatic horizontal acceleration with automatic throttle
    RTL = 6, // automatic return to launching point
    CIRCLE = 7, // automatic circular flight with automatic throttle
    LAND = 9, // automatic landing with horizontal position control
    DRIFT = 11, // semi-autonomous position, yaw and throttle control
    SPORT = 13, // manual earth-frame angular rate control with manual throttle
    FLIP = 14, // automatically flip the vehicle on the roll axis
    AUTOTUNE = 15, // automatically tune the vehicle's roll and pitch gains
    POSHOLD = 16, // automatic position hold with manual override, with automatic
throttle
    BRAKE = 17, // full-brake using inertial/GPS system, no pilot input
    THROW = 18, // throw to launch mode using inertial/GPS system, no pilot input
    AVOID_ADSB = 19, // automatic avoidance of obstacles in the macro scale - e.g. full-
sized aircraft
    GUIDED_NOGPS = 20, // guided mode but only accepts attitude and altitude
    SMART_RTL = 21, // SMART_RTL returns to home by retracing its steps
    FLOWHOLD = 22, // FLOWHOLD holds position with optical flow without rangefinder
    FOLLOW = 23, // follow attempts to follow another vehicle or ground station
    ZIGZAG = 24, // ZIGZAG mode is able to fly in a zigzag manner with predefined
point A and point B
    SYSTEMID = 25, // System ID mode produces automated system identification signals
in the controllers
    AUTOROTATE = 26, // Autonomous autorotation
    NEW_MODE = 27, // your new flight mode
};
```

2. Define a new class for the mode in [mode.h](#). It is probably easiest to copy a similar existing mode's class definition and just change the class name (i.e. copy and rename "class ModeStabilize" to "class ModeNewMode"). The new class should inherit from the Mode class and implement `run()`, `name()` and `name4()` and optionally `init()`.

```
public:
    // inherit constructor
    using Mode::Mode;
    bool init(bool ignore_checks) override;
    void run() override;

protected:
    const char *name() const override { return "NEWMODE"; }
    const char *name4() const override { return "NEWM"; }
```

The `name()` and `name4()` methods are for logging and display purposes. `init()` will be called when the vehicle first switches into this new mode so it should implement any required initialisation. `run()` will be called at 400hz and should implement any pilot input decoding and then set position and attitude targets (see below).

There are also some simple methods returning true/false that you may want to override that control features such as whether the vehicle can be armed in the new mode:

```
bool requires_GPS() const override { return false; }
bool has_manual_throttle() const override { return true; }
bool allows_arming(bool from_gcs) const override { return true; };
bool is_autopilot() const override { return false; }
```

3. Create a new mode_<new flight mode>.cpp file based on a similar mode such as [mode_stabilize.cpp](#) or [mode_loiter.cpp](#). This new file should probably implement the `init()` method which will be called when the vehicle first enters the mode. This function should return true if it is OK for the vehicle to enter the mode, false if it cannot. Below is an excerpt from [mode_rtl.cpp](#)'s init method that shows how the vehicle cannot enter RTL mode unless the home position has been set.

```
// rtl_init - initialise rtl controller
bool ModeRTL::init(bool ignore_checks)
{
    if (!ignore_checks) {
        if (!AP::ahrs().home_is_set()) {
            return false;
        }
    }
    // initialise waypoint and spline controller
    wp_nav->wp_and_spline_init();
    _state = RTL_Starting;
    _state_complete = true; // see run() method below
    terrain_following_allowed = !copter.failsafe.terrain;
    return true;
}
```

Below is an excerpt from [mode_stabilize.cpp](#)'s run method (called 400 times per second) that decodes the user's input, then sends new targets to the attitude controller.

```
void ModeStabilize::run()
{
    // convert pilot input to lean angles
    float target_roll, target_pitch;
    get_pilot_desired_lean_angles(target_roll, target_pitch, copter.aparm.angle_max,
    copter.aparm.angle_max);

    // get pilot's desired yaw rate
    float target_yaw_rate = get_pilot_desired_yaw_rate(channel_yaw->get_control_in());

    // code that sets motor spool state omitted

    // call attitude controller
    attitude_control->input_euler_angle_roll_pitch_euler_rate_yaw(target_roll,
    target_pitch, target_yaw_rate);

    // output pilot's throttle
    attitude_control->set_throttle_out(get_pilot_desired_throttle(), true,
    g.throttle_filt);
}
```

4. Instantiate the new mode class in [Copter.h](#) by searching for “ModeAcro” and then adding the new mode somewhere below.

```
Mode *flightmode;
#if MODE_ACRO_ENABLED == ENABLED
#if FRAME_CONFIG == HELI_FRAME
    ModeAcro_Heli mode_acro;
#else
    ModeAcro mode_acro;
#endif
#endif
    ModeAltHold mode_althold;
#if MODE_AUTO_ENABLED == ENABLED
    ModeAuto mode_auto;
#endif
#if AUTOTUNE_ENABLED == ENABLED
    AutoTune autotune;
    ModeAutoTune mode_autotune;
#endif
```

5. In [mode.cpp](#) add the new mode to the `mode_from_mode_num()` function to create the mapping between the mode’s number and the instance of the class.

```
// return the static controller object corresponding to supplied mode
Mode *Copter::mode_from_mode_num(const Mode::Number mode)
{
    Mode *ret = nullptr;

    switch (mode) {
        case ACRO:
            ret = &mode_acro;
            break;

        case STABILIZE:
            ret = &mode_stabilize;
            break;
```

6. Add the new flight mode to the list of valid `@Values` for the `FLTMODE1 ~ FLTMODE6` parameters in [Parameters.cpp](#) (Search for “FLTMODE1”). Once committed to master, this will cause the new mode to appear in the ground stations list of valid modes. Note that even before being committed to master, a user can setup the new flight mode to be activated from the transmitter’s flight mode switch by directly setting the FLTMODE1 (or FLTMODE2, etc) parameters to the number of the new mode.

```

// @Param: FLTMODE1
// @DisplayName: Flight Mode 1
// @Description: Flight mode when Channel 5 pwm is <= 1230
// @Values:
0:Stabilize,1:Acro,2:AltHold,3:Auto,4:Guided,5:Loiter,6:RTL,7:Circle,9:Land,11:Drift,13:Sport
// @User: Standard
GSCALAR(flight_mode1, "FLTMODE1",          FLIGHT_MODE_1),

// @Param: FLTMODE2
// @DisplayName: Flight Mode 2
// @Description: Flight mode when Channel 5 pwm is >1230, <= 1360
// @Values:
0:Stabilize,1:Acro,2:AltHold,3:Auto,4:Guided,5:Loiter,6:RTL,7:Circle,9:Land,11:Drift,13:Sport
// @User: Standard
GSCALAR(flight_mode2, "FLTMODE2",          FLIGHT_MODE_2),

```

7. Optionally you may wish to add the flight mode to the `COPTER_MODE` enum within the [mavlink/ardupilotmega.xml](https://mavlink.org/en/protocol/ardupilotmega.xml) because some ground stations may use this to automatically populate the list of available flight modes.

Scheduling Code to Run Intermittently

This page describes how you can schedule some new piece of code you have written to run intermittently.

Running your code with the scheduler

The most flexible way to run your code at a given interval is to use the scheduler. This can be done by adding your new function to the `scheduler_tasks` array in `Copter.cpp`. Note that there are actually two task lists, [the upper list](#) is for high speed CPUs (i.e. Pixhawk) and the [lower list](#) is for slow CPUs (i.e. APM2).

Adding a task is fairly simple, just create a new row in the list (higher in the list means high priority). The first column holds the function name, the 2nd is a number of 2.5ms units (or 10ms units in case of APM2). So if you wanted the function executed at 400Hz this column would contain "1", if you wanted 50Hz it would contain "8". The final column holds the number of microseconds (i.e. millions of a second) the function is expected to take. This helps the scheduler guess whether or not there is enough time to run your function before the main loop starts the next iteration.

```
/*
 scheduler table - all regular tasks apart from the fast_loop()
 should be listed here, along with how often they should be called
 (in 10ms units) and the maximum time they are expected to take (in
 microseconds)
*/
static const AP_Scheduler::Task scheduler_tasks[] PROGMEM = {
  { update_GPS,          2,    900 },
  { update_nav_mode,     1,    400 },
  { medium_loop,         2,    700 },
  { update_altitude,     10,   1000 },
  { fifty_hz_loop,       2,    950 },
  { run_nav_updates,     10,    800 },
  { slow_loop,           10,    500 },
  { gcs_check_input,     2,    700 },
  { gcs_send_heartbeat, 100,    700 },
  { gcs_data_stream_send, 2,   1500 },
  { gcs_send_deferred,   2,   1200 },
  { compass_accumulate,  2,    700 },
  { barometer_accumulate, 2,    900 },
  { super_slow_loop,     100,  1100 },
  { my_new_function,     10,    200 },
  { perf_update,         1000,   500 }
};
```

Running your code as part of one of the loops

Instead of creating a new entry in the scheduler task list, you can add your function to one of the existing timed loops. There is no real advantage to this approach over the above approach except in the case of the fast-loop. Adding your function to the fast-loop will mean that it runs at the highest possible priority (i.e. it is nearly 100% guaranteed to run at 400Hz).

- [fast_loop](#) : runs at 100Hz on APM2, 400Hz on Pixhawk
- [fifty_hz_loop](#) : runs at 50Hz
- [ten_hz_logging_loop](#): runs at 10Hz
- [three_hz_loop](#): runs at 3.3Hz
- [one_hz_loop](#) : runs at 1Hz

So for example if you want your new code to run at 10Hz you could add it to one of the case statements in the [ten_hz_logging_loop\(\)](#) function found in [Copter.cpp](#).

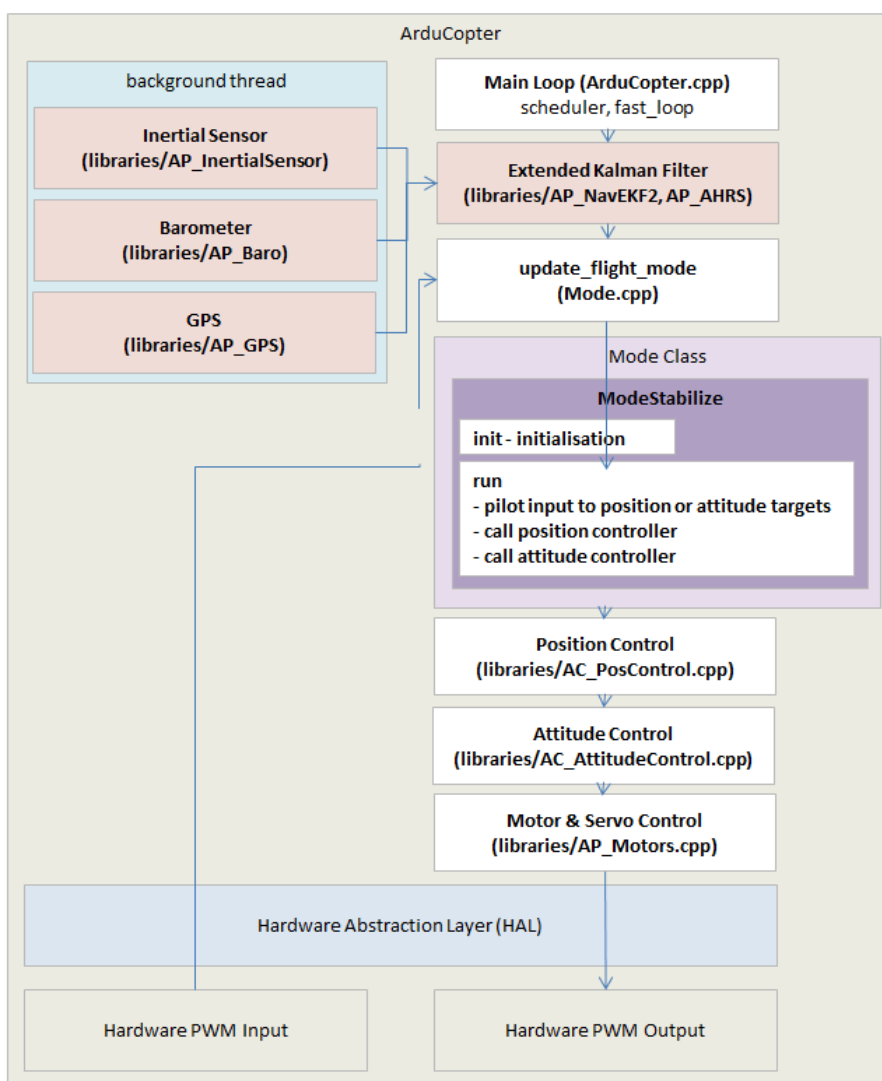
```
// ten_hz_logging_loop
// should be run at 10hz
static void ten_hz_logging_loop()
{
    if (g.log_bitmask & MASK_LOG_ATTITUDE_MED) {
        Log_Write_Attitude();
    }
    if (g.log_bitmask & MASK_LOG_RCIN) {
        DataFlash.Log_Write_RCIN();
    }
    if (g.log_bitmask & MASK_LOG_RCOUT) {
        DataFlash.Log_Write_RCOUT();
    }
    if ((g.log_bitmask & MASK_LOG_NTUN) && mode_requires_GPS(control_mode)) {
        Log_Write_Nav_Tuning();
    }

    // your new function call here
    my_new_function();
}
```

Copter Motors Library

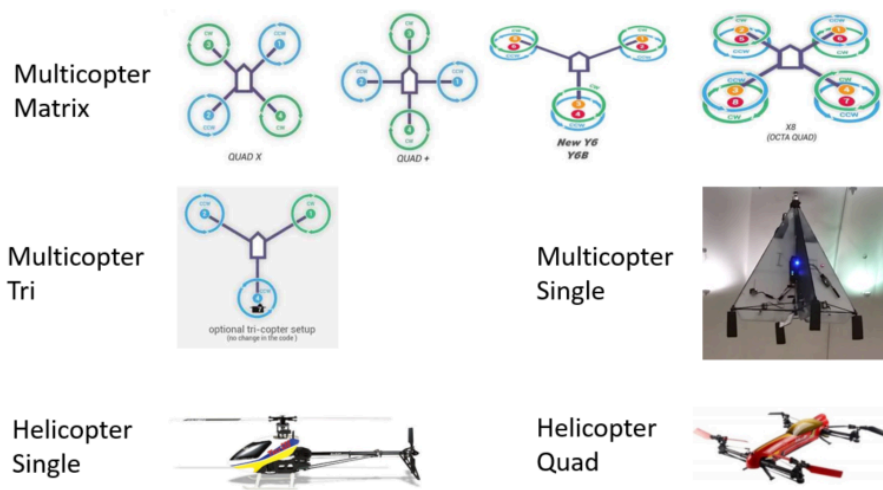
This page covers the basics of the [Copter motors library](#)

As a reference the diagram below provides a high level view of Copter's architecture. The motors library can be seen near the bottom right just above the "Hardware Abstraction Layer"



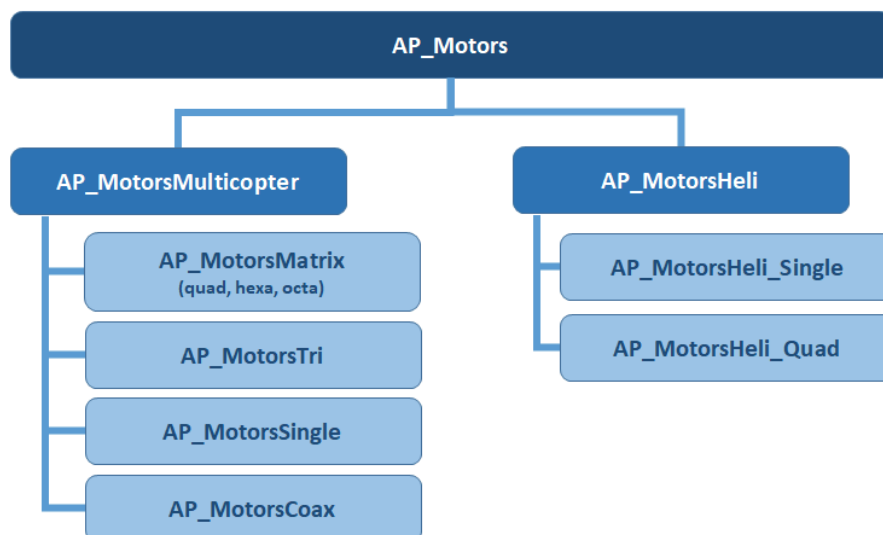
ArduPilot supports over 22 different multicopter frames. The Motors library is designed to consolidate the majority of the differences in these frame types from the higher level code including the attitude controllers and vehicle specific code. In other words the library provides a common interface so that all vehicles can be controlled in the same way without special handling being required in higher level code.

The image below shows a categorisation of these frames which is consistent with the class structure



- Multicopter Matrix : vehicles with mostly downward facing propellers and no servos. These are the most commonly used frames and the code can be found in the [AP_MotorsMatrix](#) class
- Multicopter Tri and Single: vehicles with a combination of motors and servos. These are peers of each other and Multicopter Matrix and can be found in the [AP_MotorsTri](#) and [AP_MotorsSingle](#) classes
- Helicopter Single, Quad: vehicles with swashplates that can provide reverse thrust. These are peers of each other and children of the [AP_MotorsHeli](#) class.

Below is a diagram of the Motors library's class hierarchy



Inputs to the Motors Library

The Attitude Controller provides high level roll, pitch, yaw and throttle commands to the Motors library in the range of -1 to +1 (for roll, pitch and yaw) or 0 to +1 (for throttle). The exact point where the attitude controller issues these commands can be found in AC_AttitudeControl_Multi's [rate_controller_run\(\)](#) method

```
void AC_AttitudeControl_Multi::rate_controller_run()
{
    Vector3f gyro_latest = _ahrs.get_gyro_latest();
    _motors.set_roll(rate_target_to_motor_roll(gyro_latest.x, _rate_target_ang_vel.x));
    _motors.set_pitch(rate_target_to_motor_pitch(gyro_latest.y, _rate_target_ang_vel.y));
    _motors.set_yaw(rate_target_to_motor_yaw(gyro_latest.z, _rate_target_ang_vel.z));
}
```

and the definitions of the methods used can be seen in [AP_Motors_Class.h](#)

```
// set_roll, set_pitch, set_yaw, set_throttle
void set_roll(float roll_in) { _roll_in = roll_in; }; // range -1 ~ +1
void set_pitch(float pitch_in) { _pitch_in = pitch_in; }; // range -1 ~ +1
void set_yaw(float yaw_in) { _yaw_in = yaw_in; }; // range -1 ~ +1
void set_throttle(float throttle_in) { _throttle_in = throttle_in; }; // range 0 ~ 1
```

Outputs and Responsibilities

All motors libraries must do two things:

1. Convert the high level inputs from the attitude controller to individual motor and servo outputs
 - Roll input of +1.0 should cause motor and servo outputs that provide maximum roll rotation to the right
 - Pitch input of +1.0 should result in maximum pitch back
 - Yaw input of +1.0 should result in maximum yaw in the clockwise direction
 - Throttle input of +1.0 should result in maximum acceleration upwards (in the vehicle's body frame). Throttle of zero should result in minimum acceleration upwards (or maximum acceleration downwards in the case of helicopters)
2. Set the limit flags to avoid "I-term build-up" in the attitude and throttle controllers. Although a vehicle will be flyable without setting these flags, they are important for protecting against overshoot in lean angle control and also for automatic landing detection. The limits are held in the [AP_Motors_Class::limit](#) variable.

```
// structure for holding motor limit flags
struct AP_Motors_limit {
    uint8_t roll_pitch : 1; // we have reached roll or pitch limit
    uint8_t yaw : 1; // we have reached yaw limit
    uint8_t throttle_lower : 1; // we have reached throttle's lower limit
    uint8_t throttle_upper : 1; // we have reached throttle's upper limit
} limit;
```

Adding support for a new “Multicopter Matrix” frame

To add support for a new Multicopter Matrix frame type (i.e. a frame with downward facing propellers and no servos):

- Decide on a name for the frame and add a new entry into either the `motor_frame_class` and/or `motor_frame_type` enums found in [AP_Motors/AP_Motors_Class.h](#)
- Add the same number(s) from the above step to the `FRAME_CLASS` and/or `FRAME_TYPE` parameters descriptions
- Add a new case to the `AP_MotorsMatrix::setup_motors()` for the new frame class and/or frame type
 - Use `AP_MotorsMatrix::add_motor()` or `AP_MotorsMatrix::add_motor_raw()` functions to set the roll, pitch and yaw factors for each motor on the vehicle. These factors are multiplied by the high level roll, pitch and yaw input from the attitude controllers and the result is then output to the motors.
 - The above `add_motor` function's `testing_order` argument specifies the order in which the motors should spin when the user initiates a motor test. Normally each motor's testing order should be relative to its position on the frame in a clockwise direction.
 - set `success = true` at the bottom of the case to alert the `AP_Motors` initialisation code that the frame has been setup

Detailed instructions are not yet available for adding more complex frame types (sorry!)

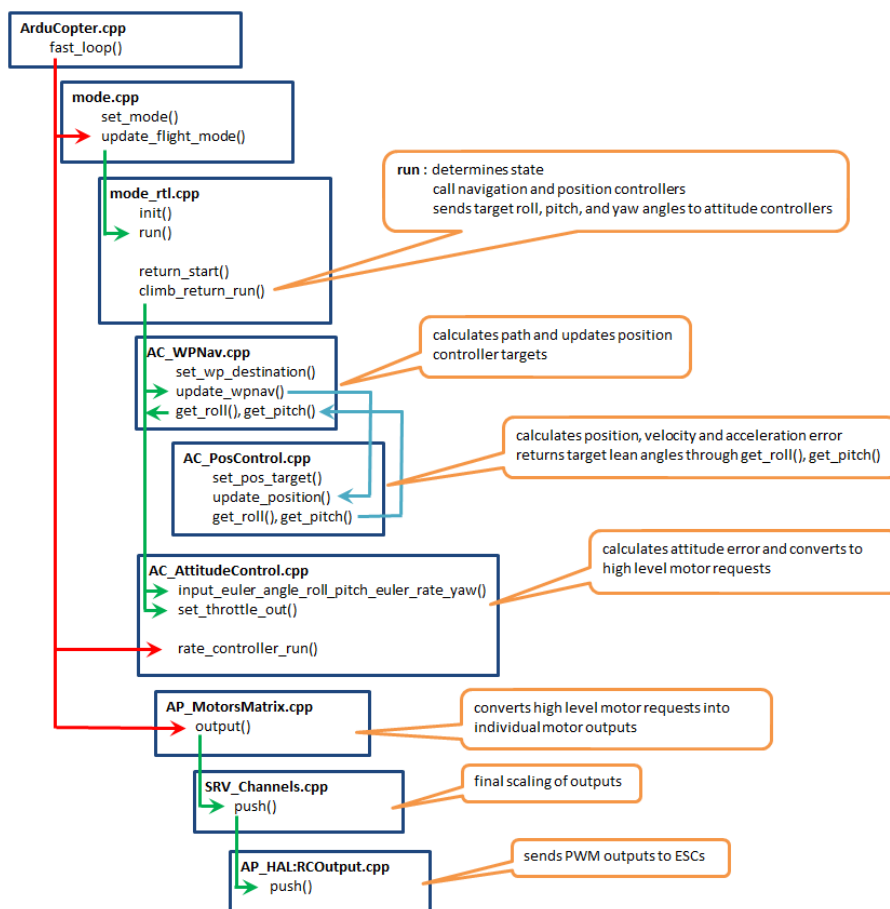
Decacopter (Arducopter custom motor mixing tests)



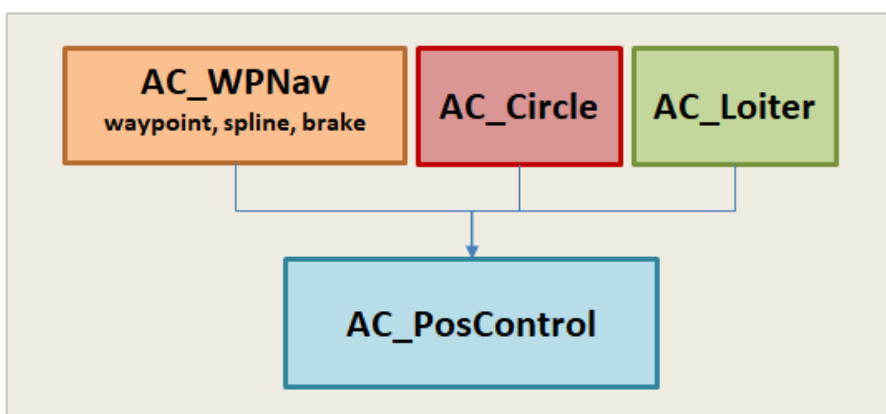
Copter Position Control and Navigation

This page covers the basics of Copter's [Position Control](#) and [Waypoint Navigation](#) libraries

As a reference the diagram below provides a high level code flow view of Copter's RTL mode

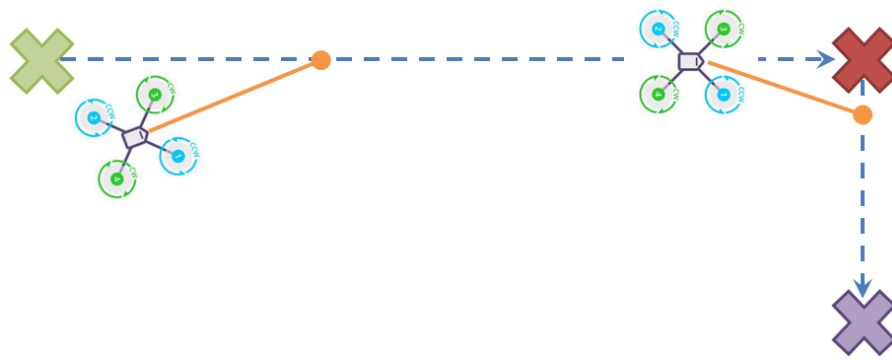


Class Hierarchy and Description

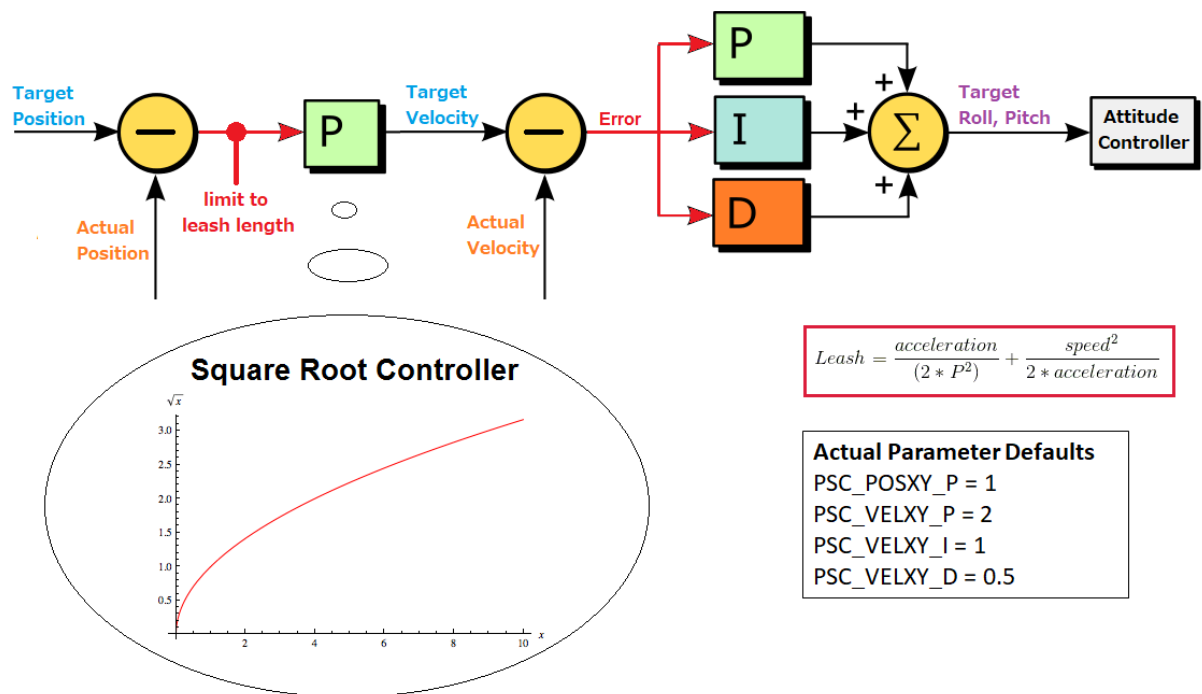


At the highest level, AC_WPNav, AC_Circle and AC_Loiter libraries are peers and all make use of the AC_PosControl library

- AC_WPNav is made up of three independent controllers (these may be separated into individual libraries in the future):
 - Waypoint attempts to fly the vehicle in a straight line to a target waypoints. This interface accepts a 3D target destination specified as a latitude, longitude and altitude or as an offset from the EKF origin
 - Spline flies the vehicle in a smooth curved path to a 3D target waypoint with a final velocity that will allow it to continue smoothly towards a subsequent waypoint. Its interface is very similar to the Waypoint controller above
 - Brake tries to slow the vehicle to a stop as quickly as possible
 - Waypoint, Spline and Brake controllers do not use PIDs directly. Instead they update target positions or velocities which are then passed to the Position Controller



- AC_PosControl:
 - Separate interfaces for horizontal (X and Y axis) control and vertical (Z-axis) control. These interfaces are separated because some flight modes (like AltHold mode) only require the Z-axis controller
 - Layered PID controllers are used
 - XY axis uses a Position P to convert position error to a target velocity. A velocity PID converts velocity error into a desired acceleration which is then converted to a desired lean angle which is then sent into the attitude control library.



- The Z axis uses a Position P controller to convert position error to a target vertical velocity (aka climb rate). A Velocity P controller converts velocity error to a desired acceleration. An Acceleration PID converts acceleration error into a desired throttle which is then sent into the attitude control library (which mostly just passes it through to the low level motors library)
- AC_PosControl also includes a 3D velocity controller and a 3D Position+Velocity controller

Should I use PosControl or WPNav?

Because both AC_PosControl and AC_WPNav allow the caller to move a vehicle to a 3D target point it may not be clear which should be used.

In general the vehicle code (i.e. flight mode code) should use the AC_WPNav library because it will ensure the vehicle flies a straight path to the target. AC_PosControl should normally only be used by other higher level navigation libraries. The exception to this rule is simpler flight modes that simply want Z-axis control - these make direct use of AC_PosControl.

Copter Object Avoidance

ArduCopter, from release 3.5, supports object avoidance using a [Lightware SF40C](#), [TeraRanger Tower](#) or with any sensor capable of providing distances using the MAVLink [DISTANCE_SENSOR](#) message. This page describes how the object avoidance feature works and how “proximity sensors” should provide data into ArduPilot.

Avoidance in Loiter

Object Avoidance with SF40c



The video above shows a test of an EnRoute EX700 copter fitted with a [Lightware SF40C 360degree lidar](#). The vehicle stops 2m from the fence regardless of the speed of the vehicle towards the fence. This is possible because ArduPilot knows the speed of the vehicle (using the GPS and accelerometers), the distance to the barrier and the maximum acceleration/deceleration of the vehicle.

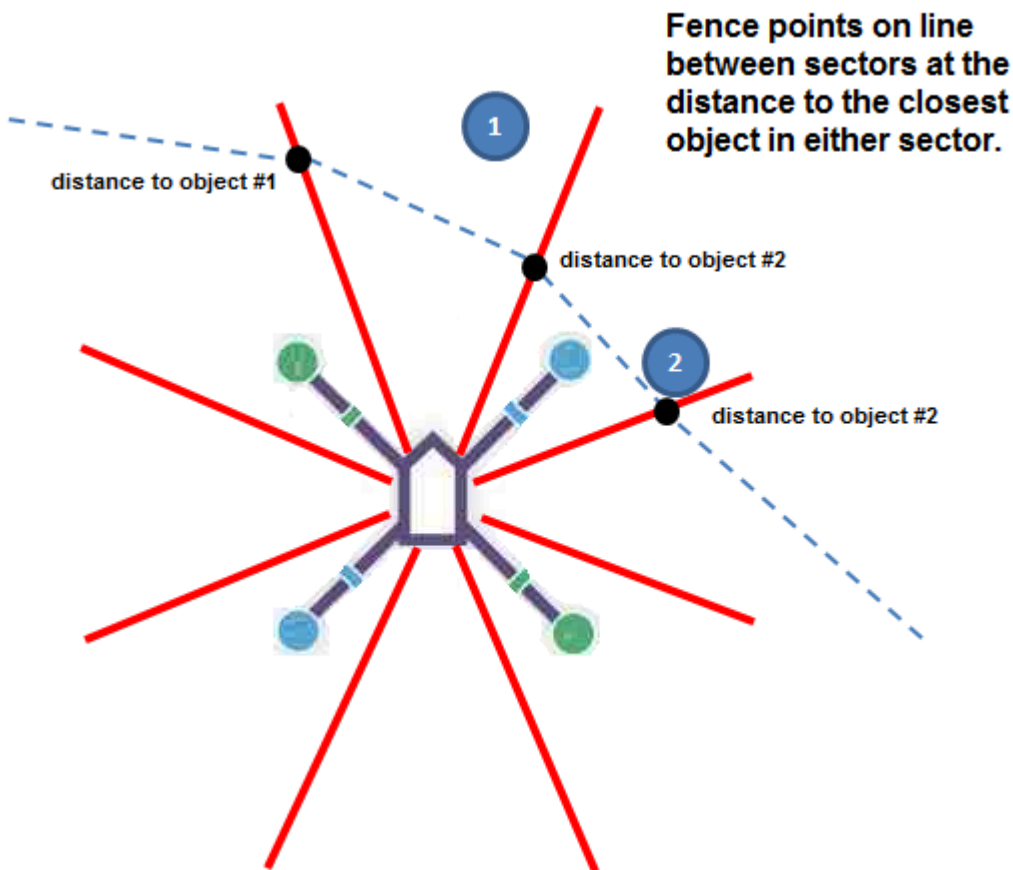
The [AP_Proximity library](#) collects the raw distance measurements from the sensor. These raw distance measurements are consolidated so that only the closest distance and angle within 8 sectors is stored and used.

Each sector is 45 degrees wide with sector 0 pointing forward of the vehicle, sector 1 is forward-right, etc. From these distances and angles a fence (an array of 2D Vectors) is built up around the vehicle. The fence points fall on the lines between sectors at a conservative

distance.

! Note

The default is to divide the area around the vehicle into 8 sectors but each proximity sensor driver can override this and use a different number of sectors.



mini fence method

The [AC_Avoidance library](#) consumes the above fence and uses it to adjust (mostly shortens) desired velocity vectors sent to it by the [Loiter controller](#) (search for “avoid->adjust_velocity”). Loiter’s desired velocity vectors come from the pilot’s inputs.

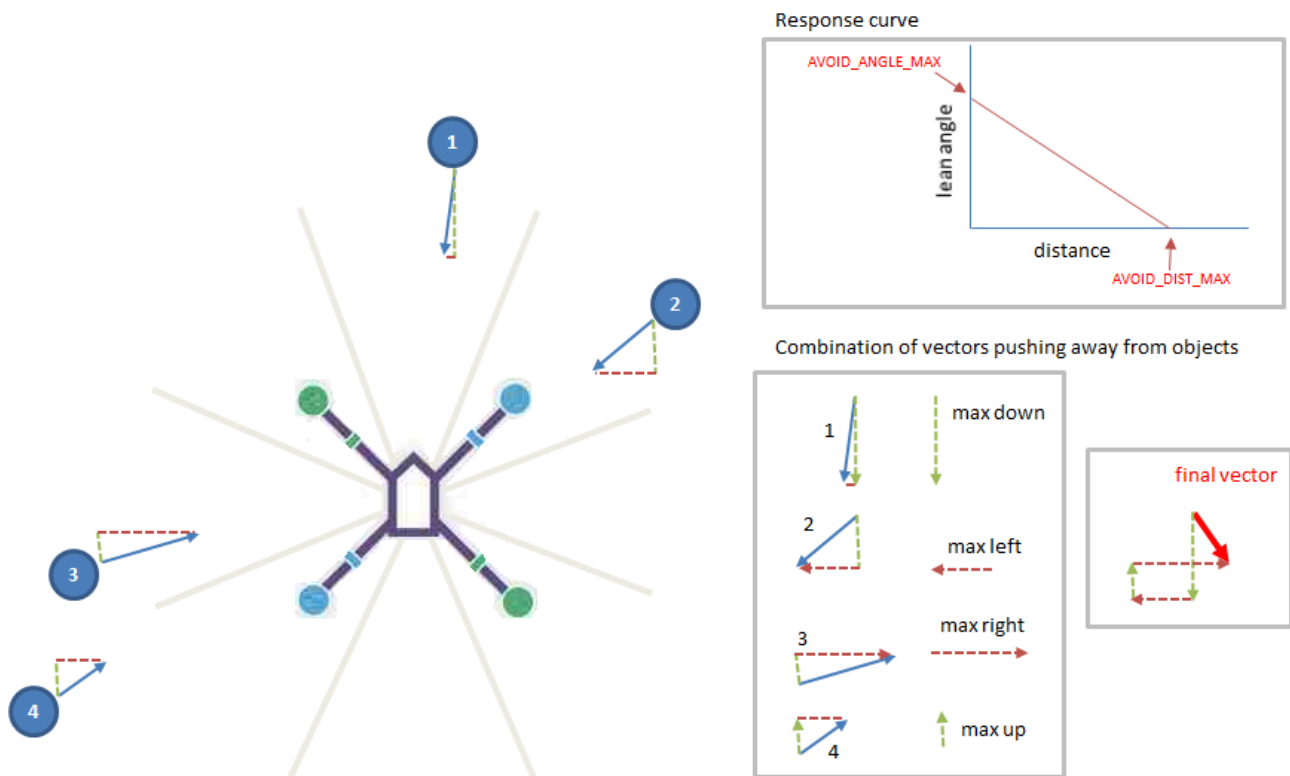
! Note

Object avoidance also works for Guided mode’s velocity controller

Avoidance in AltHold

The [AP_Proximity](#) and [AC_Avoidance libraries](#) are also used for object avoidance in AltHold. Because AltHold does not rely upon the GPS, the vehicle’s velocity towards the barriers is not known and a much simpler version of object avoidance is used in which the distance to the barrier is converted into a lean angle.

The image below shows the method by which the distances to various objects are combined together to result in a final roll/pitch lean angle. Each sensed object's distance and angle from the vehicle is turned into a roll/pitch angle. The largest positive and negative roll and pitch angles are found and combined together resulting in a final roll/pitch angle.



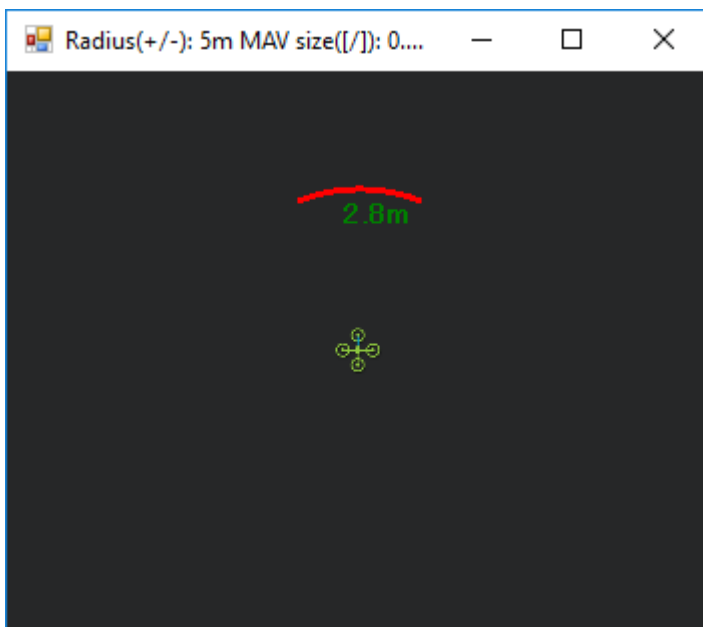
This final roll/pitch angle is combined with the pilot's input meaning the vehicle tends to fly away from objects but the pilot always maintains the ability to override the object avoidance. This is quite different from Loiter mode in which the pilot cannot force the vehicle to fly into an object.

! Note

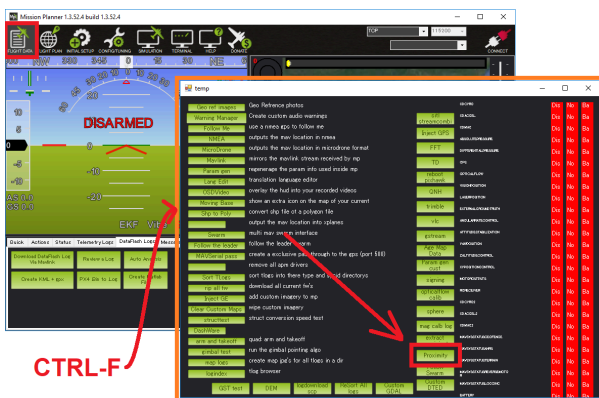
The vehicle will also stop before hitting barriers above it if there is an upward facing range finder. Currently this range finder's distance must be sent to ardupilot using the [DISTANCE_SENSOR](#) message with the orientation field set to 24 (upwards).

Reporting to the Ground Station

MissionPlanner, from v1.3.48 onwards (and hopefully other ground stations in the future), shows the distance to nearby objects in a RADAR type window.



This window can be opened by going to the Flight Data screen, press Ctrl-F and push the Proximity button. Values will appear if the [PRX_TYPE](#) parameter is set to enable a sensor, and messages are being received.



This is accomplished by the vehicle sending `DISTANCE_SENSOR` messages to the ground station at regular intervals. The method that does this can be seen in the [GCS_Common.cpp](#) file (search for “send_proximity”). Note that this means that it is possible that `DISTANCE_SENSOR` messages can be used both to send distances into the vehicle code, and then similar `DISTANCE_SENSOR` messages might be sent out from the vehicle to the ground station. These are separate messages though and are likely transferred at a different rate.

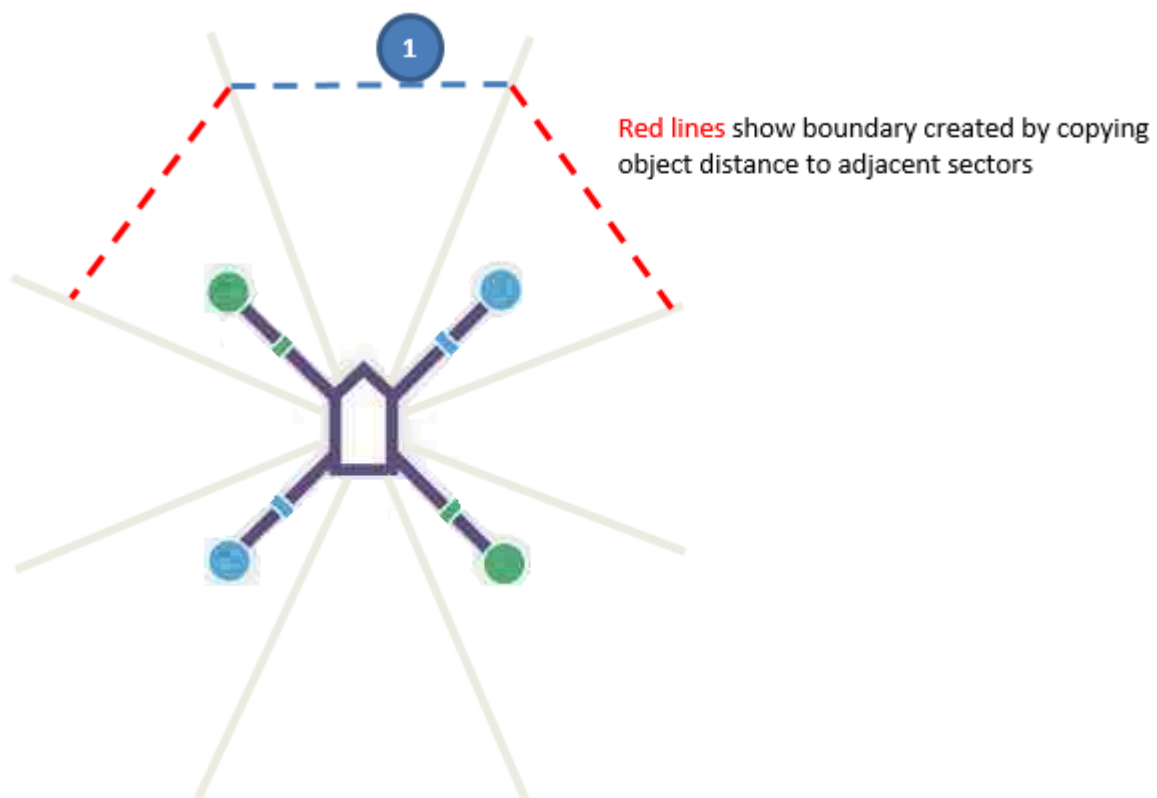
Providing Distance Sensor messages to ArduPilot

For developers of new “proximity” sensors (i.e. sensors that can somehow provide the distance to nearby objects) the easiest method to get your distance measurements into ardupilot is to send `DISTANCE_SENSOR` message for each direction the sensor is capable of. The system id of the message should match the system id of the vehicle (default is “1” but can be changed using the `SYSID_THISMAV` parameter). The component id can be anything but `MAV_COMP_ID_PATHPLANNER` (195) or `MAV_COMP_ID_PERIPHERAL` (158) are probably good choices.

These messages should be sent at between 10hz and 50hz (the faster the better). The fields should be filled in as shown below:

- time_boost_ms : 0 (ignored)
- min_distance : the minimum distance that the sensor can measure in centimeters (i.e. 100 = 1m). This number should generally not change and should be the same regardless of the orientation field.
- max_distance : the maximum distance that the sensor can measure in centimeters (i.e. 1500 = 15m). This number should generally not change and should be the same regardless of the orientation field.
- current_distance : the shortest distance in cm to the object
- type : 0 (ignored)
- id : 0 (ignored)
- orientation : 0 to 7 (0=forward, each increment is 45degrees more in clockwise direction), 24 (upwards) or 25 (downwards). search for MAV_SENSOR_ORIENTATION on the [mavlink/common page](https://mavlink.io/en/common.html).
- covariance : 0 (ignored)

When DISTANCE_SENSOR messages are not received for all 8 sectors, empty sectors are filled in with the distance from an adjacent sector (if available). This conveniently leads to a “cup” shaped fence which is more likely to protect the vehicle from hitting the object. It is likely this will be changed in future releases especially if the “stop” behaviour (instead of “slide” behaviour) is configured.



These two videos shows a [ZED 3D camera](#) connected to an [NVidia TX1](#) running [OpenKai](#) and providing forward facing distance measurements to ArduPilot using the method described above.

ArduPilot Object Avoidance with 3D camera



ArduPilot Object Avoidance with 3D camera (onboard video)



Future Steps

- add avoidance to other modes including RTL, AUTO ([issue here](#))
- improve integration with ROS ([issue here](#))
- implement a simple occupancy grid for cases where we cannot (or do not want to) implement ROS on the vehicle ([issue here](#))

If you would like to get involved with the development of this area, please consider posting in the issues listed above or chatting with the developers on [ArduPilot Discord](#).

Adding Custom Attitude Controller to Copter

Custom controller library allows you to implement and easily run your controller inside ArduPilot in a systematic way. Original-primary-mains means existing controller and custom-secondary means new controller. This library aimed to not interfere with other parts of the main controller or vehicle level code. The controller output is sent to the control allocation library, known as the mixer, the same way the main controller does. The custom controller library has the following features:

Features

- In-flight switching between main and custom controller with RC switch, option 109.
- Bitmask to choose which axis to use the custom controller output
- Filter, integrator reset mechanism when switching between controller - Bumpless transfer when switching from custom to the main controller
- Ground and in-flight spool state separation to avoid build-up during arming and take-off with the custom controller
- Frontend-backend separation that allows adding a new controller with very little overhead
- Flag to compile out custom controller-related code on hardware, `-enable-custom-controller`
- Proper parameter table implementation that allows adding new custom controller parameter table without corruption
- Single parameter to switch between different custom controllers, reboot required
- Multiple checks to avoid accidentally running miss-configured, or unconfigured, custom controller with RC switch
- Custom controller parameters start with `CC_` in the GCS.

Parameters

The frontend library has the following parameters

- **CC_TYPE**: choose which custom controller backend to use, reboot required. - Setting it to 0 will turn this feature off, GCS will not display parameters related to the custom controller
- **CC_AXIS_MASK**: choose which axis to use custom controller output - This is a bitmask type parameter. Set 7 to use all output

! Warning

The custom controller requires the user to set up the build environment ([Building the code](#)) and cloned the ArduPilot GitHub repo locally ([Downloading the Code / Using Git](#)). The example controller is for experimental purposes only and it has been tested only lightly. The user is advised to use caution while testing it on a real vehicle.

Interaction With Main Controller

The custom controller update function is called after the main rate controller is run and before the motor output library is called. This placement allows overriding motor library mixer input, namely `_roll_in`, `_pitch_in`, and `_yaw_in` values, without causing functional change inside the motor library. This is the same way the main controller sends its output to the control allocation library, known as a mixer. This reduces latency to the minimum level.

The custom controller library uses the same target attitude as the main controller. Most of the code inside the `AC_AttitudeControl` library is related to input shapings such as how to interpret pilot commands depending on the flight mode or high-level position controller output. For example, in STABILIZE mode pilot RC commands are scaled based on maximum lean angle and yaw rate parameters, these values are passed to `input_euler_angle_roll_pitch_euler_rate_yaw` function. The pilot command is fed into a first-order input shaping algorithm to smooth out any jitter due to the RC link and to generate an acceleration-limited attitude target. Later, attitude controller `attitude_controller_run_quat` is called, an attitude error is calculated based on acceleration limited `_attitude_target` value, and target rate is generated. Even if acceleration limiting is turned off by setting `ATC_RATE_FF_ENAB` to 0, the target attitude is still presented with an `_attitude_target` variable.

Every input shaping function inside `AC_AttitudeControl` calls `attitude_controller_run_quat` at the end to run the attitude controller, except when flying in ACRO mode with the `acro` option set to rate only. Even in this case `_attitude_target` is updated properly and the pilot command can be accessed via the `rate_bf_targets` function.

By default, the input shaping algorithm is turned on. This produces kinematically consistent attitude target and rate feed-forward values. Take a look at the custom PID backend to see how rate feed-forward is added to the attitude controller output. It is advised to use the rate feed-forward value if `ATC_RATE_FF_ENAB` is enabled, otherwise the pilot might feel significant lag between RC commands and vehicle responses.

After running the custom controller, mixer input is sent to the motor library via `set_roll`, `set_pitch`, `set_yaw` functions.

Bumpless Transfer

When switching from the custom controller to the main controller it is important to reset the main controller target, error, d-term filters, and set each axis integrator properly. Otherwise, a sudden jump in the controller error signal or motor output will be observed, which could result in a jerky motion. To allow a smooth transition between controllers, the main controller reset function is called when switching out of the custom controller which reset all three axes of the main controller. The reset is performed inside `reset_main_atti_controller` function.

The attitude target and rate target are also made equal to the current attitude and gyro rate to make the error signal grow from zero in the `relax_attitude_controllers` function. To avoid impulse input to the controller, the target resetting is not performed when the feedforward is disabled by setting `ATC_RATE_FF_ENAB` parameter to 0.

```
void AC_CustomControl::reset_main_att_controller(void)
{
    // reset attitude and rate target, if feedforward is enabled
    if (_att_control->get_bf_feedforward()) {
        _att_control->relax_attitude_controllers();
    }

    _att_control->get_rate_roll_pid().set_integrator(0.0);
    _att_control->get_rate_pitch_pid().set_integrator(0.0);
    _att_control->get_rate_yaw_pid().set_integrator(0.0);
}
```

Backend Type

Currently, there 2 custom controller backends available. These are

- Empty backend set `CC_TYPE = 1`
- PID backend set `CC_TYPE = 2`

Empty Controller - `CC_TYPE = 1`

The empty controller does not do any calculations. It is created to make it easier to copy and implement your new controller. The main controller is not reset when switching from an empty controller.

PID Controller - `CC_TYPE = 2`

The PID controller backend has the same controller architecture as the main controller. It doesn't have any safeguarding mechanism such as acceleration limiting or rate limiting. The default gains are scaled 0.9 times to differentiate the custom controller response from the main one. Since this controller does not have acceleration limiting, specifically a square root

controller, it would be safer to give a gentle command while flying with it. Although it has the same architecture as the main one, a proper reset functionality is not implemented intentionally to make it easier to detect the effect of improper resetting.

How To Use It

The custom controller is enabled by default in SITL. You can test it using the PID backend.

Step #1: Compile and run the default SITL model. In the GCS, choose the custom controller type, assign an axis mask and set which RC switch to activate the custom controller. Reboot autopilot. For example in MAVProxy, using RC channel 6 as the enable/disable switch for the custom controller:

```
param set CC_TYPE 2
param set CC_AXIS_MASK 7
param set RC6_OPTION 109
reboot
```

Step #2: Run the following command to display backend parameters. These would be under `CC2_` for the PID backend.

```
param set CC2*
```

Step #3: Arm and take off. While hovering, switch RC6 to high. In MAVProxy, you can do this with

```
rc 6 2000
```

Step #4: You should be prompted with `Custom controller is ON` message on GCS to indicate that the custom controller is running.

Step #5: Set RC6 to low to switch back to the main controller. You should be prompted with `Custom controller is OFF` message on GCS.

Real Flight Testing

It is recommended that you always arm, take-off, land, and disarm while the main controller is running. You should switch to the custom controller while the vehicle is hovering steadily. This will reduce the effect of improper filter resetting. You should arm and take off with the custom controller only if proper ground idling is implemented.

To test it on hardware compile with the `--enable-custom-controller` flag.

```
./waf configure --board CubeOrange copter --enable-custom-controller
```

Post Flight Logs

Switching in and out of the custom controller is logged under the “CC” in the log tree. You can see the time and duration the custom controller is activated under “CC.Act”

How To Add New Custom Controller

You can add your custom controller backend with the following steps which are demonstrated in this video. Let's assume we are adding the 3rd custom controller.

ArduCopter Custom Controller Part 1: Create New Backend



Step #1: Generate a copy of `AC_CustomControl_Empty.cpp` and `AC_CustomControl_Empty.h` within `AC_CustomControl` folder. The folder tree would look like this,

```
AC_CustomControl.cpp
AC_CustomControl.h
AC_CustomControl_Backend.h
AC_CustomControl_Empty.cpp
AC_CustomControl_Empty.h
AC_CustomControl_Empty - Copy.cpp
AC_CustomControl_Empty - Copy.h
.
.
.
```

PID and README files are omitted to keep it simple.

Step #2: Change `Empty - Copy` suffix with your own choice, let's called it `XYZ`, which would look like

```
AC_CustomControl.cpp
AC_CustomControl.h
AC_CustomControl_Backend.h
AC_CustomControl_Empty.cpp
AC_CustomControl_Empty.h
AC_CustomControl_XYZ.cpp
AC_CustomControl_XYZ.h
.
.
.
```

Step #3: Change every class name, function definition etc. from `AC_CustomControl_Empty` to `AC_CustomControl_XYZ` inside `AC_CustomControl_XYZ.cpp` and `AC_CustomControl_XYZ.h` files. This would look like this for the header file

```
#pragma once

#include "AC_CustomControl_Backend.h"

class AC_CustomControl_XYZ : public AC_CustomControl_Backend {
public:
    AC_CustomControl_XYZ(AC_CustomControl &frontend, AP_AHRS_View*& ahrs,
        AC_AttitudeControl_Multi*& atti_control, AP_MotorsMulticopter*& motors, float dt);

    Vector3f update(void) override;
    void reset(void) override;

    // user settable parameters
    static const struct AP_Param::GroupInfo var_info[];

protected:
    // declare parameters here
    AP_Float param1;
    AP_Float param2;
    AP_Float param3;
};
```

Step #4: Increase the maximum number of custom control variables by one and update the custom control type enum

from

```
#define CUSTOMCONTROL_MAX_TYPES 2
```

to

```
#define CUSTOMCONTROL_MAX_TYPES 3
```

```
enum class CustomControlType : uint8_t {  
    CONT_NONE          = 0,  
    CONT_EMPTY         = 1,  
    CONT_PID           = 2,  
    CONT_XYZ           = 3,  
};  
    // controller that should be used
```

Step #5: Add a new backend header in `AC_CustomControl.cpp` file. Place it under other backend includes.

```
#include "AC_CustomControl_Backend.h"  
#include "AC_CustomControl_Empty.h"  
#include "AC_CustomControl_PID.h"  
#include "AC_CustomControl_XYZ.h"
```

Step #6: Add new backend parameter in `AC_CustomControl.cpp` file. Increment `_backend_var_info` array index, backend parameter prefix and parameter table index by one. Place it under the other backend's parameters.

```
.  
.   
.   
    // parameters for empty controller  
    AP_SUBGROUPVARPTR(_backend, "1_", 6, AC_CustomControl, _backend_var_info[0]),  
  
    // parameters for PID controller  
    AP_SUBGROUPVARPTR(_backend, "2_", 7, AC_CustomControl, _backend_var_info[1]),  
  
    // parameters for XYZ controller  
    AP_SUBGROUPVARPTR(_backend, "3_", 8, AC_CustomControl, _backend_var_info[2]),  
  
    AP_GROUPEND  
};
```

Step #7: Allow creating new backend class in `AC_CustomControl.cpp` file inside init function.

```

.
.
.
case CustomControlType::CONT_PID:
    _backend = new AC_CustomControl_PID(*this, _ahrs, _atti_control, _motors, _dt);
    _backend_var_info[get_type()] = AC_CustomControl_PID::var_info;
    break;
case CustomControlType::CONT_XYZ:
    _backend = new AC_CustomControl_XYZ(*this, _ahrs, _atti_control, _motors, _dt);
    _backend_var_info[get_type()] = AC_CustomControl_XYZ::var_info;
    break;
default:
    return;
}

```

Step #8: Add the custom controller in `AC_CustomControl_config.h` file.

```

#ifndef AP_CUSTOMCONTROL_XYZ_ENABLED
#define AP_CUSTOMCONTROL_XYZ_ENABLED AP_CUSTOMCONTROL_BACKEND_DEFAULT_ENABLED
#endif

```

Step #9: This is the bare minimum to compile and run your custom controller. You can add controller-related code to the `AC_CustomControl_XYZ` file without changing anything else.

Step #10: You can add new parameters by following the instructions in the [Adding a parameter to a library](#) wiki page.

Step #11: Initialize the class object in the backend's constructor. For example in PID backend

```

// put controller-related variable here

// angle P controller objects
AC_P          _p_angle_roll2;
AC_P          _p_angle_pitch2;
AC_P          _p_angle_yaw2;

// rate PID controller objects
AC_PID _pid_atti_rate_roll;
AC_PID _pid_atti_rate_pitch;
AC_PID _pid_atti_rate_yaw;

```

above P or PID classes are initialized in the backend's constructors,

```

AC_CustomControl_PID::AC_CustomControl_PID(AC_CustomControl &frontend, AP_AHRS_View* &ahrs,
AC_AttitudeControl_Multi* &atti_control, AP_MotorsMulticopter* &motors, float dt) :
    AC_CustomControl_Backend(frontend, ahrs, atti_control, motors, dt),
    _p_angle_roll2(AC_ATTITUDE_CONTROL_ANGLE_P * 0.90f),
    _p_angle_pitch2(AC_ATTITUDE_CONTROL_ANGLE_P * 0.90f),
    _p_angle_yaw2(AC_ATTITUDE_CONTROL_ANGLE_P * 0.90f),
    _pid_atti_rate_roll(AC_ATC_MULTI_RATE_RP_P * 0.90f, AC_ATC_MULTI_RATE_RP_I * 0.90f,
AC_ATC_MULTI_RATE_RP_D * 0.90f, 0.0f, AC_ATC_MULTI_RATE_RP_IMAX * 0.90f,
AC_ATC_MULTI_RATE_RP_FILT_HZ * 0.90f, 0.0f, AC_ATC_MULTI_RATE_RP_FILT_HZ * 0.90f, dt),
    _pid_atti_rate_pitch(AC_ATC_MULTI_RATE_RP_P * 0.90f, AC_ATC_MULTI_RATE_RP_I * 0.90f,
AC_ATC_MULTI_RATE_RP_D * 0.90f, 0.0f, AC_ATC_MULTI_RATE_RP_IMAX * 0.90f,
AC_ATC_MULTI_RATE_RP_FILT_HZ * 0.90f, 0.0f, AC_ATC_MULTI_RATE_RP_FILT_HZ * 0.90f, dt),
    _pid_atti_rate_yaw(AC_ATC_MULTI_RATE_YAW_P * 0.90f, AC_ATC_MULTI_RATE_YAW_I * 0.90f,
AC_ATC_MULTI_RATE_YAW_D * 0.90f, 0.0f, AC_ATC_MULTI_RATE_YAW_IMAX * 0.90f,
AC_ATC_MULTI_RATE_YAW_FILT_HZ * 0.90f, AC_ATC_MULTI_RATE_YAW_FILT_HZ * 0.90f, 0.0f, dt)
{
    AP_Param::setup_object_defaults(this, var_info);
}

```

Step #12: Add your controller inside update function of `AC_CustomControl_XYZ.cpp` file. This function returns a 3-dimensional vector consisting of roll, pitch, yaw, and mixer input, respectively.

Step #13: Add reset functionality inside reset function of `AC_CustomControl_XYZ.cpp` file. It is the user's responsibility to add proper controller resetting functionality. This highly depends on the controller and it should not be copy-pasted from another backend without testing it in SITL.

Step #14: You can access target attitude and target rate values using `atti_control->get_attitude_target_quat()` and `atti_control->rate_bf_targets()` functions, respectively. You can also access latest gyro measurement using `_ahrs->get_gyro_latest()` function.

Preparing Your Simulink Model For Code Generation

Simulink model of the controller needs to be configured before the code generation. These steps are demonstrated in the following video.

ArduCopter Custom Controller Part 2 : SimuLink Controller Code Gener...



Step #1: In Solver tab in Configuration, change solver type to Fixed-Type. This is mandatory for code generation to work. Change Solver to discrete(no continuous state) this will reduce generated code's flash cost and run-time load. Change Fixed-step size to one over the [SCHED_LOOP_RATE](#). The code generation needs to be repeated each time this parameter is changed.

Step #2: In the Hardware Implementation tab, change Device vendor to ARM Compatible and Device type to ARM Cortex-M. This determines which data types are available and what is their size.

Step #3: In the Code Generation tab, change the System target file to ert.tlc Embedded Coder and language to C++. Since we don't want Simulink to compile the generated code click Generate code only. Click Package code and artifacts to get only relevant generated code files in a single folder.

Step #4: In Code Generation->Optimization tab, change Default parameter behavior to Tunable. This will create a variable inside the generated code with the same name as they are defined in Matlab or Simulink workspace.

Step #5: To force generated code to be strictly single precision, change Inport and Outport block data type to single. Since mathematical operation in ArduPilot runs in single precision.

Step #6: In Code Mapping->Data, change Inport and Outport data visibility to public, and member access method to none. This will allow passing input and output as an argument.

Step #7: In Code Mappings->Data, change Model parameters data visibility to public. This will allow accessing model parameters inside the custom controller backend.

Step #8: In Code Mappings->Functions, change step function input and output argument type. If these are changed to pointer as shown in the video, step 6 is mandatory. The choice of how to pass input and output depends on the user's preference. Inputs are passed by pointer and outputs passed by reference for simplicity.

Step #9: The last step is to click on generate code button to start code generation. If a problem arises during this process, Simulink will throw an error message to explain the problem and how to fix it.

There are a lot of options in Simulink code generation in terms of class, function, variable naming, code generation optimization level, parameters access and storage method, etc. It is up to the user to change these depending on their project needs.

Please watch the following video to understand what each generated header and cpp files contain.

How to Understand Code Generated from Simulink



Integration Generated Code Into ArduPilot

The generated source code needs to be copied inside the ArduPilot and called inside the newly created custom controller backend. The steps to achieve this is demonstrated in the following video.

ArduCopter Custom Controller Part 3: Integration of Controller Code in...



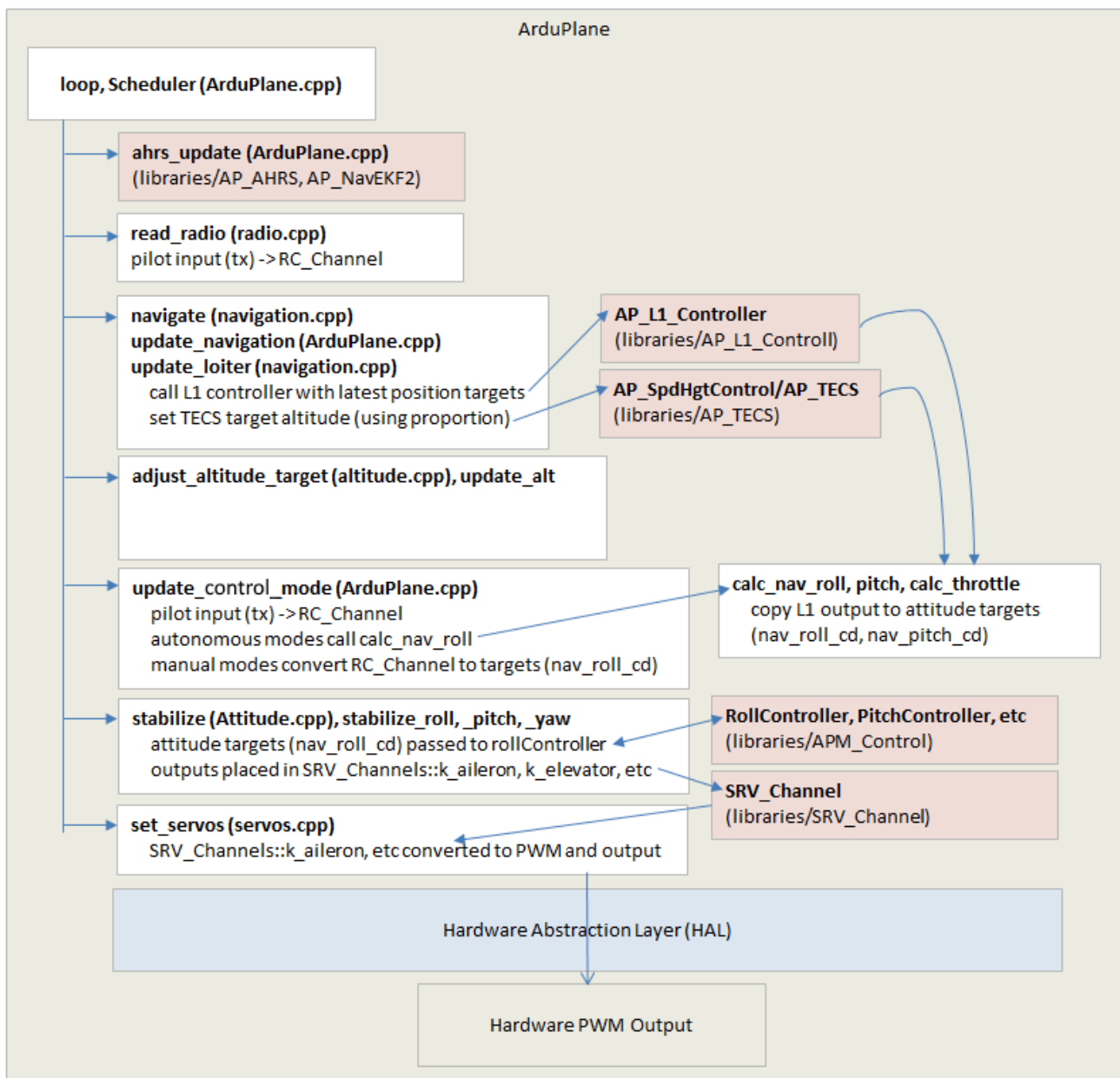
Step #1: Copy content of zip file into a new folder inside ardupilot/libraries. Call AC_Simulink as in the example. **Step #2:** Tell waf to build this folder if the custom controller is enabled. **Step #3:** Create an object of simulink code generated class. **Step #4:** Call initialize function of this object inside backend constructor. **Step #5:** Call step function of this object inside backend update function. Arrange input and output arguments by following the similar pattern as in ert_main.cpp. **Step #6:** Once these steps are completed we can compile ArduPilot.

As long as the input and output type and how they are passed to the step function remain the same, the steps given in here and in the previous section do not need to be repeated each time we do code generation. Even if we change the controller type for example from P+PID to PID+PID, we just need to click on the generate code button and copy and paste the zip file content to the AC_Simulink folder.

An example of this implementation is given in this PR
<https://github.com/ArduPilot/ardupilot/pull/21700>.

Plane Architecture Overview

This page attempts to roughly show the ArduPlane architecture. Please note it was not written by Plane experts so there may be some inaccuracies.



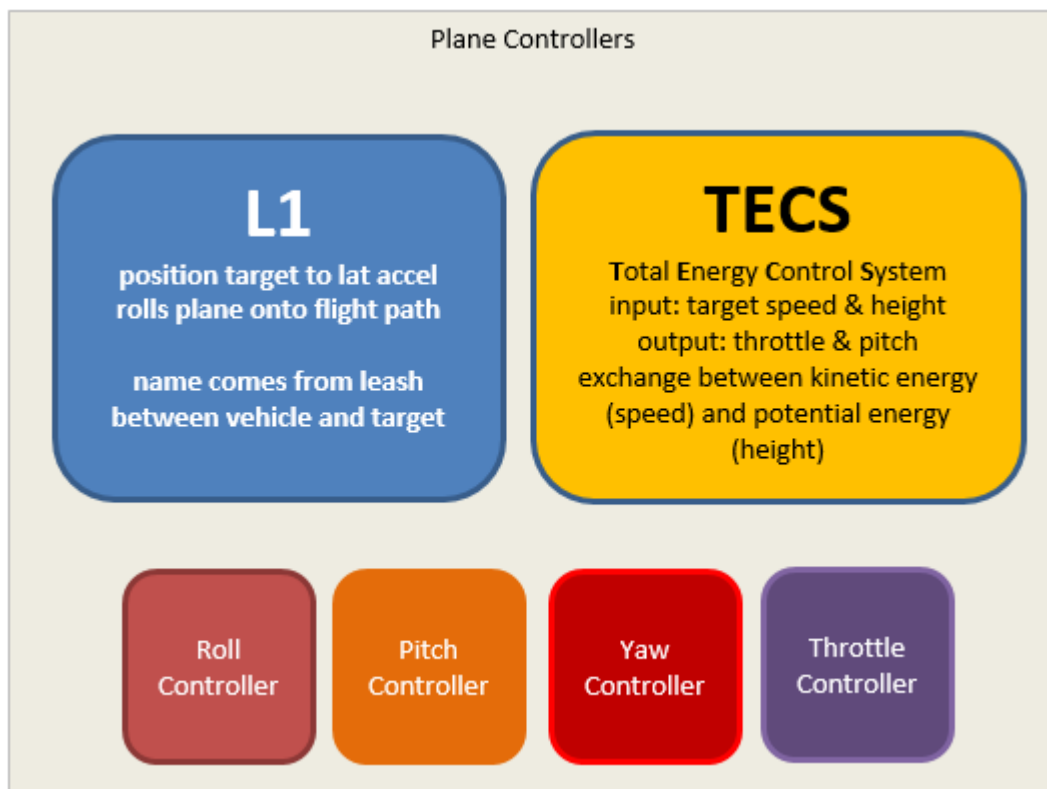
Above is a high level diagram of ArduPlane's architecture. One large difference between Plane vs Copter and Rover is flight modes are not organised into a single file (like they are in Copter) or class (like they are in Rover). Instead the code path goes through stages. Each stage then decides what it should do based on the flight mode.

These stages can be seen in the diagram above and include:

- `ahrs_update` calls the EKF to consume the latest sensor data and produce a attitude and position estimate.
- `read_radio` reads in the pilot's input and calculates the appropriate attitude or position targets
- `navigate` calls the L1 and TECS controllers (see below) to convert position targets for the roll, pitch and throttle controllers
- `update_control_mode` copies the L1 controller's roll and pitch targets to `nav_roll_cd` and `nav_pitch_cd` global variables
- `stabilize` executes the lower level roll, pitch and throttle controllers
- `set_servos` sends the output from the roll, pitch and throttle controllers to the appropriate servo outputs

Plane Controllers

Plane has 2 high level controllers and at least 3 lower level controllers shown below.



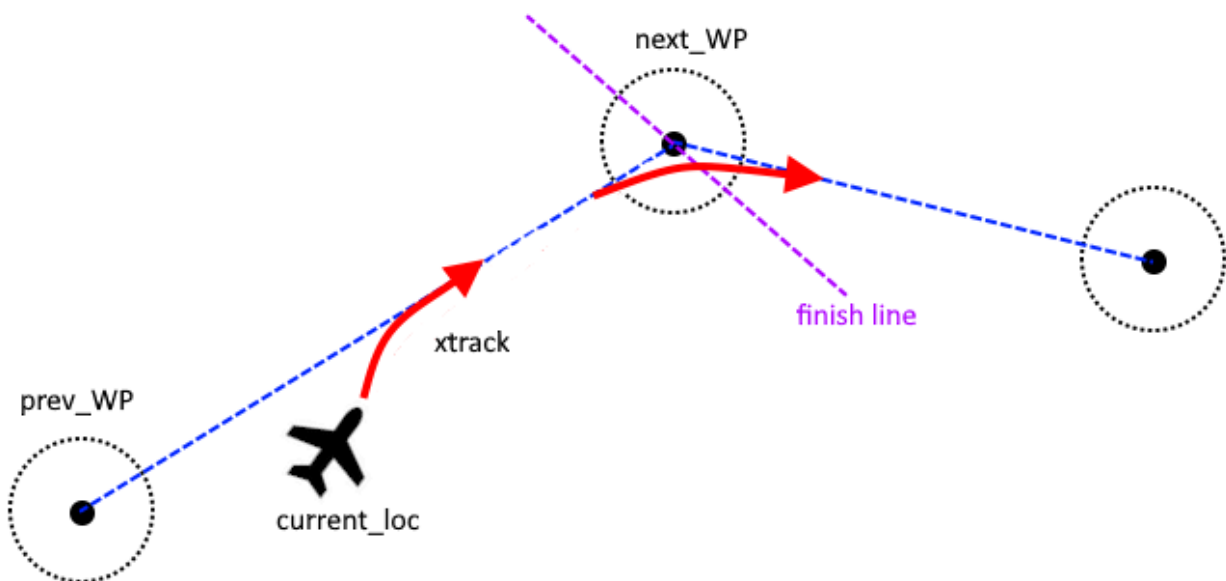
- the L1 controller converts a origin and destination (each expressed as a latitude, longitude) into a lateral acceleration to make the vehicle travel horizontally along the path from the origin to the destination.
- the TECS (Total Energy Conservation System) controls the exchange between the vehicle's kinetic energy (i.e. speed) and its potential energy (i.e. altitude). Its inputs are a target speed and height and it attempts to reach these targets by calculating target throttle and pitch values which are then passed to the lower level pitch and throttle controllers.

Plane Navigation and Altitude Control Overview

Navigation

In auto-navigating modes like AUTO and CRUISE, or when responding to MAVLink navigation commands, ArduPlane usually uses two WAYPOINT locations (see [location object](#)) and flies a wind compensated path between them:

- `next_WP`: the destination
- `prev_WP`: the beginning point



The code usually uses `do_xxx` to setup these waypoints (see [commands logic module](#)). Sometimes, rather than having an existing starting point to use as `prev_WP`, code can use the `set``next_WP``` method using `auto_state.next_WP_crosstrack` variable value to set the `prev_WP` to either the current waypoint destination (`next_WP`), or current location. Usually, you want to have the crosstrack false for the first waypoint so you proceed directly from your current position, instead of tracking to the line from a previously set destination. When entering a new mode, the `prev_WP` is always set to the current location, and crosstracking is initially disabled to assure navigation directly to whatever the new `next_WP` is set by the mode.

Auto-navigating modes call their `navigation method`, usually involving a `verify_xxx` command, to determine if the destination has been reached and setup the next destination, if needed. Determining course correction and navigation roll angles is done by the [L1 Controller](#).

[AP_Common:Location.h](#) has many useful methods for manipulation locations.

Altitude Control

Altitude control is managed by [TECS \(Total Energy Control System\) for Speed and Height Tuning Guide](#) using demanded altitude from the mode, in altitude controlled mode. Modes use the `update_target_altitude` to calculate demanded altitude for TECS which will calculate the altitude profile between waypoints([setup_glide_slope](#) or maintain current altitude. Altitude target can be changed by MAVLink command (DO_CHANGE_ALT) or by stick control using `update_fbwb_speed_height` method.

Note

if you are actually moving away from the next waypoint, as usually the case if RTL climbing before turning toward home option is set, the `prev_WP` will have to be continuously reset to present position and the glide slope re-setup in order to continue to climb.

Adding a new Log Message

[DataFlash Logs](#) are stored on the flight controller's onboard dataflash memory and can be downloaded after a flight. The logs provide detailed information about each flight which can be important especially when trying to [diagnose why something went wrong](#).

This page explains how to write a new dataflash log message.

⚠ Note

a guide to understanding the formats of data logger messages is [HERE](#) and how the code for these messages is parsed and checked for inclusion in the documentation is [HERE](#)

The Easy Way

Use [AP::logger\(\).Write\(...\)](#):

```
void Write(const char *name, const char *labels, const char *fmt, ...);  
void Write(const char *name, const char *labels, const char *units, const char *mults,  
const char *fmt, ...);
```

An example of the top function being used can be found in [Compass_learn.cpp](#):

```
AP::logger().Write("COFS", "TimeUS,OfsX,OfsY,OfsZ,Var,Yaw,wVar,N", "QffffffI",  
                    AP_HAL::micros64(),  
                    best_offsets.x,  
                    best_offsets.y,  
                    best_offsets.z,  
                    best_error,  
                    best_yaw_deg,  
                    worst_error,  
                    num_samples);
```

- the 1st argument is the message name. This should be 4 characters or less and be unique
- the 2nd argument is a comma separated list of up to 16 field names; with a limit of 64 characters in total

- the 3rd argument is a format string (maximum of 16 characters) with each letter holding the format for the corresponding field. The meaning of each letter can be found [here in AP_Logger/LogStructure.h](#)
- the remaining arguments are the actual values that will be logged.

The 2nd Log_Write function is the same as the first except that it accepts two additional string arguments, “units” and “mults”. Similar to the “format” argument, each character in these arguments specifies the [units](#) (i.e. “d” for degrees) or [multiplier](#) (i.e. “2” for *100, “B” for *0.01) for the following fields. These help the graphing tools scale the output correctly when displaying to the user.

For example, below is a “TEST” log message which outputs the current system time and altitude.

```
AP::logger().Write("TEST", "TimeUS,Alt",
    "sm", // units: seconds, meters
    "FB", // mult: 1e-6, 1e-2
    "Qf", // format: uint64_t, float
    AP_HAL::micros64(),
    alt_in_cm);
```

- the time has units of seconds (“s”), multiplier of “F” to indicate the value should be divided by 1 million and format of “Q” to indicate it is output as a 64 bit unsigned integer.
- the altitude has units of meters (“m”), multiplier of “B” to indicate the value should be divided by 100 (to convert from centimeters to meters) and format of “f” because the value is a float.

The Harder Way

For commonly used messages, especially those which are output at a relatively high rate (50hz or more) a slightly more efficient logging method can be used.

- decide if the logging will come from a library (i.e. the call to Log_Write will from somewhere within the [libraries directory](#)) or the vehicle code (i.e. from within [ArduCopter](#), [ArduPlane](#), etc)
- add a new entry to the [vehicle’s #define list or enum in defines.h](#) or [AP_Logger/LogStructure.h’s LogMessages enum](#).
- define a structure to hold the values to be logged in either the vehicle’s [Log.cpp](#) file or in [AP_Logger/LogStructure.h](#). All log file messages should have time_us as their first field.

```
struct PACKED log_Test {
    LOG_PACKET_HEADER;
    uint64_t time_us;
    float a_value;
}
```

- add the log message's name, [units](#), [multiplier](#) and [format](#) strings into the [vehicle's LogStructure](#) array or [AP_Logger/LogStructure.h's LOG_EXTRA_STRUCTURES](#) array
- add a new method to the vehicle code or AP_Logger library called `Write_<something-or-other>` which fills in the structure and then calls [AP_Logger/WriteBlock\(\)](#)

```
void Copter::Log_Write_Test()
{
    struct log_Test pkt = {
        LOG_PACKET_HEADER_INIT(LOG_TEST_MSG),
        time_us   : AP_HAL::micros64(),
        a_value   : 1234
    };
    logger.WriteBlock(&pkt, sizeof(pkt));
}
```

- call this new function from the [scheduler](#) or from some other place in the code at the moment you wish to log the values

Adding a new MAVLink Message

Data and commands are passed between the ground station (i.e. Mission Planner, QGroundControl, MAVProxy, etc.) using the [MAVLink protocol](#) over a serial interface. This page provides some high-level advice for adding a new MAVLink message.

These instructions have only been tested on Linux (to be precise a VM running Ubuntu on a Windows machine). Instructions for setting up a VM are on the [SITL page](#). If you can run SITL, you should be able to follow the advice here. These instructions will not run natively on Windows or a Mac.

Step #1: Ensure you have the latest ArduPilot code installed. Also, check mavproxy. Mavproxy can be updated by running this command in a Terminal window:

```
sudo pip install --upgrade mavproxy
```

Step #2: Decide what type of message you want to add and how it will fit in with the existing [MAVLink messages](#).

For example, you might want to send a new navigation command to the vehicle so that it can perform a trick (like a flip) in the middle of a mission (i.e. in AUTO mode). In this case, you would need a new MAV_CMD_NAV_TRICK similar to the [MAV_CMD_NAV_WAYPOINT](#) definition (search for “MAV_CMD_NAV_WAYPOINT” in the [MAVLink messages](#) page).

Alternatively, you may want to send down a new type of sensor data from the vehicle to the ground station. Perhaps similar to the [SCALED_PRESSURE](#) message.

Step #3: Add the new message definition to the [common.xml](#) or [ardupilotmega.xml](#) file in the MAVLink submodule.

If this command is generally useful, and will hopefully be added to the MAVLink protocol, then it should be added to the `../modules/mavlink/message_definitions/v1.0/common.xml` file. If it is only for your personal use or only applicable to ArduPilot, then it should be added to the `ardupilotmega.xml` file.

Step #4: Add functions to the main vehicle code to handle sending or receiving the command to/from the ground station. A compile will be needed (ie. `./waf copter`) to generate the MAVLink packet code so make sure to do that after editing the XML file. The MAVLink generation happens first, so it doesn't matter if the project compilation is successful or not due to other source code changes.

The top-level of this code will most likely be in the vehicle's `GCS_MAVLink_<vehicle>.cpp` file (`GCS_MAVLink_Copter.cpp` for ArduCopter) or in the `../libraries/GCS_MAVLink/GCS` class.

In the first example where we want to add support for a new navigation command (i.e., a trick) the following would be required:

- Extend the `AP_Mission` library's `mission_cmd_to_mavlink()` and `mavlink_to_mission_cmd()` functions to convert the MAVProxy command into an `AP_Mission::Mission_Command` structure.
- Add a new case to the vehicle's `commands_logic.cpp` 's (`mode_auto.cpp` for ArduCopter) `start_command()` and `verify_command()` functions to check for the arrival of the new `MAV_CMD_NAV_TRICK`. These should call two new functions that you create called `do_trick()` and `verify_trick()` (see below).
- Create these two new functions, `do_trick()` and `verify_trick()`, that somehow command the vehicle *to perform the trick* (perhaps by calling another function in `control_auto.cpp` (`mode_auto.cpp` for ArduCopter) that sets the `auto_mode` variable and then calls a new `auto_trick_start()` function). The `do_trick()` function will be called when the command is first invoked. The `verify_trick()` will be called at 10hz (or higher) repeatedly until the trick is complete. The `verify_trick()` function should return true when the trick has been completed.

Step #5: Decide how you are going to handle the message at the GCS. One of the simplest ways is to use Mavproxy. MavProxy uses pymavlink to define the MAVLink messages, so you will need to rebuild pymavlink to include your custom message.

- Remove the currently installed version of pymavlink. `pip uninstall pymavlink`
- Install the updated version. CD to `ardupilot/modules/mavlink/pymavlink` and run `python3 setup.py install --user`
- Mavproxy is now capable of sending or receiving the new message. To ask it to print out or send your message you need to implement a module. Modules are python plugins that allow you to add functionality to Mavproxy. By default on Ubuntu they are located in `/usr/local/lib/python2.7/dist-packages/MAVProxy/modules/`. Here is an example of a module that prints the contents of a `MY_CUSTOM_PACKET` message. Look at the other modules for examples on how to trigger sending of messages using the command line interface.

```
#!/usr/bin/env python
'''Custom'''

import time, os

from MAVProxy.modules.lib import mp_module
from pymavlink import mavutil
import sys, traceback

class CustomModule(mp_module.MPModule):
    def __init__(self, mpstate):
        super(CustomModule, self).__init__(mpstate, "Custom", "Custom module")
        '''initialisation code'''

    def mavlink_packet(self, m):
        'handle a MAVLink packet'
        if m.get_type() == 'MY_CUSTOM_PACKET':
            print "My Int: %(x).2f" % \
                {"x" : m.intField}

def init(mpstate):
    '''initialise module'''
    return CustomModule(mpstate)
```

⚠ Warning

If the message you added has an ID greater than 255 you will need to enable MAVLink 2 support. This can be done by setting the relevant `SERIALn_PROTOCOL` parameters (e.g. `SERIAL1_PROTOCOL`) to 2 and starting Mavproxy with the `--mav20` argument.

Step #6: Consider contributing your code back to the main code base. Discuss this with other developers on [Discord](#) and/or [raise a pull request](#). If you raise a pull request it is best to separate the change into at least two separate commits. One commit for the changes to the .xml files (i.e Step #3) and another for the changes to the vehicle code.

Adding Support for a new MAVLink Gimbal

This page covers how ArduPilot interacts with a MAVLink enabled gimbal. It is meant more as a guide on how gimbal manufacturers can make their gimbals work with ArduPilot with minimum code changes to ArduPilot .

The [SToRM32 gimbal](#) is a good reference as it supports MAVLink.

Messages the gimbal should support

1. The gimbal should listen on the serial port for a [HEARTBEAT](#) message from the vehicle. It can generally assume that the first heart beat it receives will be from the vehicle but to be certain you can check the “type” field to be sure it’s something sensible (i.e. MAV_TYPE = 1 for fixed wing, 2 for quadcopters but 6 for GCSs should be ignored).

This heart beat will contain the vehicle’s system-id and component-id. The gimbal should adopt the same system-id but should use a component-id of MAV_COMP_ID_GIMBAL (i.e. 154) for all its future messages. In fact, any unique component-id can be used as long as it’s not zero nor the component id of the vehicle or any other device on the vehicle.

2. The gimbal should send a [HEARTBEAT](#) message out the serial port at approximately 1hz
 - the system-id and component-id should be as mentioned above.
 - “type” should be MAV_TYPE_GIMBAL (i.e. 26).
 - “autopilot” is not used so can be set to anything including MAV_AUTOPILOT_GENERIC (i.e. 0)
 - “base_mode” and “custom_mode” are not used so can be set to anything (perhaps 0 is best)
 - “system_status” should be set to “MAV_STATE_ACTIVE” once the gimbal is ready to accept attitude targets
3. To support reading/writing parameter values from the ground station the gimbal should implement these message:
 - [PARAM_REQUEST_READ](#) - if the gimbal receives this message and “target_system” and “target_component” values match the gimbal’s system-id and component-id, it should respond with a [PARAM_VALUE](#) message which contains the value of the parameter specified by the “param_id” field (simply an enum, the gimbal can assign whatever enum it wishes to each of its internal parameters)
 - [PARAM_REQUEST_LIST](#) - respond to this message by sending a [PARAM_VALUE](#) message for every parameter within the gimbal.

- [PARAM_SET](#) - respond to this by setting the internal variable to the value in the "param_value" field.
4. ArduPilot will send angle requests to the gimbal via MAVLink which will arrive as [COMMAND_LONG](#) messages with the "command" field set to MAV_CMD_DO_MOUNT_CONTROL (i.e 205).
- Param #1 contains desired pitch in degrees
 - Param #2 contains desired roll in degrees
 - Param #3 contains desired yaw in degrees but note that the yaw is in relation to the front of the vehicle so "0" is straight ahead, "90" is to the right, "-90" is to the left.
5. If the gimbal needs extra data from the vehicle it can request it using the [REQUEST_DATA_STREAM](#) message.
- the target system and component id should be for the vehicle.
 - "req_stream_id" can be any values in the MAV_DATA_STREAM enum. Some useful values are:
 - MAV_DATA_STREAM_POSITION will cause the vehicle to send [GLOBAL_POSITION_INT](#) messages which includes lat, lon, alt, velocity (3d) and heading)
 - MAV_DATA_STREAM_EXTRA1 will send the [MSG_ATTITUDE](#) which includes euler angles for roll, pitch, yaw

Testing

To test the system, you should be able to connect the gimbal to a Pixhawk as if it's a STORM32 gimbal. In particular check the "[Set-up through the Mission Planner \(MAVlink protocol\)](#)" section.